

The Ultimate Multivariate Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	3
2	Part 0 — The 30-Second Exam Strategy	4
3	Part 1 — Foundations	4
3.1	1.1 What is multivariate data?	4
3.2	1.2 Why not just run p univariate tests?	4
3.3	1.3 Notation	4
3.4	1.4 The sample covariance matrix	5
3.5	1.5 Mean vector test (intuition)	5
4	Part 2 — Preparing Multivariate Data	5
4.1	2.1 A basic multivariate data frame	5
4.2	2.2 Extract the numeric matrix	6
4.3	2.3 Mean vector and covariance matrix	6
4.4	2.4 Standardize (when units differ)	7
4.5	2.5 Group summary (for comparison designs)	7
5	Part 3 — Two-Sample Hotelling T^2	7
5.1	3.1 Model	7
5.2	3.2 Manual implementation	8
5.3	3.3 Which variables drive the difference	9
5.4	3.4 Confidence ellipse for the mean difference	9
5.5	3.5 Assumptions	10
6	Part 4 — MANOVA (3+ Groups)	11
6.1	4.1 Model	11
6.2	4.2 Fit MANOVA	11
6.3	4.3 Follow-up univariate ANOVAs	12
6.4	4.4 Post-hoc on group means (per response)	12
6.5	4.5 Assumptions	13
7	Part 5 — Multivariate Normality	14
7.1	5.1 Why it matters	14
7.2	5.2 What MVN looks like	14
7.3	5.3 Mardia's test (manual)	14
7.4	5.4 Chi-square Q-Q plot for MVN	15
7.5	5.5 Practical decision	16
7.6	5.6 Report template	16
8	Part 6 — Covariance Structures	16

8.1	6.1 Why model the covariance?	16
8.2	6.2 Compound symmetry (CS)	17
8.3	6.3 Testing sphericity	17
8.4	6.4 Compound symmetry vs unstructured	18
8.5	6.5 When to use each structure	18
9	Part 7 — Principal Component Analysis	19
9.1	7.1 What PCA does	19
9.2	7.2 Running PCA	19
9.3	7.3 Eigenvalues and variance explained	19
9.4	7.4 How many components to keep?	20
9.5	7.5 Loadings (component equations)	21
9.6	7.6 Naming components	22
9.7	7.7 Orthogonality check	22
9.8	7.8 Component scores	22
9.9	7.9 Group comparison on scores	23
9.10	7.10 Biplot	23
9.11	7.11 Covariance vs correlation PCA	25
9.12	7.12 Reporting template	25
10	Part 8 — Choosing Number of Components (Formal Methods)	26
10.1	8.1 Kaiser criterion	26
10.2	8.2 Broken-stick model	26
10.3	8.3 Cumulative variance threshold	26
10.4	8.4 Parallel analysis (manual approximation)	27
11	Part 9 — Factor Analysis	27
11.1	9.1 What FA does	27
11.2	9.2 Fit factor analysis	27
11.3	9.3 Reading the output	28
11.4	9.4 Choosing the number of factors	29
11.5	9.5 Rotation	29
11.6	9.6 Bartlett's test of sphericity	30
11.7	9.7 KMO (Kaiser-Meyer-Olkin) — sampling adequacy	30
11.8	9.8 Reporting template	31
12	Part 10 — Discriminant Analysis (LDA, QDA)	31
12.1	10.1 What it does	31
12.2	10.2 Fit LDA	31
12.3	10.3 Predictions and confusion matrix	32
12.4	10.4 QDA	33
12.5	10.5 Testing LDA assumptions	33
12.6	10.6 Reporting template	33
13	Part 11 — Cluster Analysis	34
13.1	11.1 Hierarchical clustering	34
13.2	11.2 K-means clustering	35
13.3	11.3 Silhouette coefficient	36
13.4	11.4 Comparing methods	37
13.5	11.5 Reporting template	37
14	Part 12 — Canonical Correlation Analysis (CCA)	37
14.1	12.1 What it does	37
14.2	12.2 Fit CCA	37
14.3	12.3 Test significance	38

14.4	12.4 Reporting template	38
15	Part 13 — Multivariate Outliers	39
15.1	13.1 Mahalanobis distance	39
15.2	13.2 Robust covariance	39
15.3	13.3 Diagnostic plot	39
16	Part 14 — Testing Sphericity and Compound Symmetry	40
16.1	14.1 Sphericity test (Mauchly)	40
16.2	14.2 Compound symmetry test	41
17	Part 15 — Theory Q&A	41
18	Part 16 — Exam Scenarios	43
18.1	16.1 “Are the two groups different on all responses jointly?”	43
18.2	16.2 “MANOVA with 3+ groups”	43
18.3	16.3 “Test multivariate normality”	43
18.4	16.4 “Run PCA and interpret”	43
18.5	16.5 “How many components?”	43
18.6	16.6 “Factor analysis with k factors”	43
18.7	16.7 “Discriminant analysis”	43
18.8	16.8 “Cluster the observations”	44
18.9	16.9 “Detect outliers”	44
18.10	16.10 “Test compound symmetry / sphericity”	44
19	Part 17 — Quick Reference	44
20	Part 18 — Common Mistakes	45
21	Part 19 — The Single Most Important Idea	45
21.1	What is covered	46

1 How to Use This Cheat Sheet

Every multivariate question falls into one of these. Find your question, jump to the section.

Question type	Section	Typical phrasing
Compare two mean vectors	3	“Do the two groups differ in mean vector?”
Compare 3+ mean vectors	4	“MANOVA for group differences”
Test multivariate normality	5	“Is the data MVN?”
Estimate covariance structure	6	“Compound symmetry / unstructured”
Reduce dimension	7	“Run PCA and interpret”
Number of components	8	“How many components to keep?”
Latent factors	9	“Factor analysis / Bartlett test”
Classify observations	10	“Discriminant analysis”
Cluster observations	11	“Hierarchical / k-means clustering”
Correlation between two sets	12	“Canonical correlation”
Detect multivariate outliers	13	“Mahalanobis distance”
Check sphericity	14	“Test for sphericity”
Theory questions	15	“What is Hotelling T ² ?”

Question type	Section	Typical phrasing
Exam scenarios	16	Mixed
Quick reference	17	—
Common mistakes	18	—

Golden rule of this document: every chunk is self-contained.

2 Part 0 — The 30-Second Exam Strategy

For **any** multivariate question, run this pipeline:

1. **Identify the goal:** compare means, reduce dimensions, find structure, classify, cluster?
2. **Extract the numeric matrix** `X <- as.matrix(df %>% select(numeric_vars))`.
3. **Standardize** if variables have different units (`scale(X)`).
4. **Fit the right method:**
 - Two groups, mean vectors → Hotelling T^2
 - 3+ groups, mean vectors → MANOVA
 - Distribution → Mardia's test
 - Dimension reduction → `prcomp` (PCA) or `factanal` (FA)
 - Classification → `MASS::lda` / `MASS::qda`
 - Clustering → `hclust` / `kmeans`
 - Outliers → Mahalanobis distance
5. **Report** eigenvalues, loadings, test statistic, p-value.
6. **Interpret** components/factors by naming them from the loadings.

If you can do these six steps, you can answer any multivariate question.

3 Part 1 — Foundations

3.1 1.1 What is multivariate data?

Each observation is a **vector** of p responses: $\mathbf{y}_i = (y_{i1}, \dots, y_{ip})^\top$.

- **Univariate:** one response per unit.
- **Multivariate:** multiple responses per unit, potentially correlated.

Examples: measurements on people (height, weight, BP), scores on tests (math, verbal, science), gene expression.

3.2 1.2 Why not just run p univariate tests?

1. **Inflated Type I error** — p tests inflate family-wise error to $1 - (1 - \alpha)^p$.
2. **Ignores correlation** — univariate tests ignore the covariance between responses, losing power.
3. **Doesn't answer the joint question** — “do groups differ on the *vector* of responses?” is different from “do groups differ on *any* response?”

3.3 1.3 Notation

Symbol	Meaning
$\mathbf{y}_i$	response vector for unit i ($p \times 1$)
$\mathbf{X}$	$n \times p$ data matrix
$\bar{\mathbf{y}}$	sample mean vector ($p \times 1$)
$\mathbf{S}$	sample covariance matrix ($p \times p$)
$\mathbf{R}$	sample correlation matrix ($p \times p$)
μ	population mean vector
Σ	population covariance matrix
$\mathbf{S}_p$	pooled covariance matrix

3.4 1.4 The sample covariance matrix

$$\mathbf{S} = \frac{1}{n-1} \sum_{i=1}^n (\mathbf{y}_i - \bar{\mathbf{y}})(\mathbf{y}_i - \bar{\mathbf{y}})^\top$$

- Diagonal entries: variances.
- Off-diagonal entries: covariances.
- $\mathbf{R}$ = correlation matrix = $\mathbf{D}^{-1/2} \mathbf{S} \mathbf{D}^{-1/2}$ where $\mathbf{D}$ = diag of $\mathbf{S}$.

3.5 1.5 Mean vector test (intuition)

To test $\mathbf{H}_0 : \mu = \mu_0$ (one-sample):

$$T^2 = n(\bar{\mathbf{y}} - \mu_0)^\top \mathbf{S}^{-1}(\bar{\mathbf{y}} - \mu_0)$$

Converted to F: $F = \frac{n-p}{p(n-1)} T^2 \sim F_{p, n-p}$.

Interpretation: Large $T^2 \rightarrow$ mean vector differs from μ_0 .

4 Part 2 — Preparing Multivariate Data

4.1 2.1 A basic multivariate data frame

```
# --- Self-contained: multivariate data frame ---
set.seed(201)
p2_n <- 120
p2_df <- tibble(
  id      = 1:p2_n,
  gender = factor(rep(c("M", "F"), each = 60)),
  q1 = rnorm(p2_n, 50, 10),
  q2 = rnorm(p2_n, 55, 12),
  q3 = rnorm(p2_n, 48, 9),
  q4 = rnorm(p2_n, 60, 11),
  q5 = rnorm(p2_n, 52, 10),
  q6 = rnorm(p2_n, 58, 13)
)
head(p2_df)

## # A tibble: 6 x 8
##   id gender    q1    q2    q3    q4    q5    q6
##   <int> <fct>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      1 M      52.9  67.3  51.1  63.7  45.8  84.2
## 2      2 M      46.6  45.3  52.4  35.1  47.0  32.0
## 3      3 M      53.3  57.7  49.8  74.0  61.6  58.3
## 4      4 M      33.0  48.5  46.7  45.3  47.8  78.6
## 5      5 M      37.1  83.8  45.7  69.3  53.6  55.9
## 6      6 M      49.2  24.1  46.3  58.7  71.3  51.3
```

4.2 2.2 Extract the numeric matrix

```
# --- Self-contained: numeric matrix ---
set.seed(202)
p2_df <- tibble(
  id = 1:100, g = factor(rep(c("A", "B"), each = 50)),
  q1 = rnorm(100), q2 = rnorm(100), q3 = rnorm(100)
)

p2_X <- p2_df %>% select(starts_with("q")) %>% as.matrix()
dim(p2_X)
```

```
## [1] 100  3
```

```
class(p2_X)
```

```
## [1] "matrix" "array"
```

Why: Almost every multivariate method needs a numeric matrix, not a data frame with factors.

4.3 2.3 Mean vector and covariance matrix

```
# --- Self-contained: mean and covariance ---
set.seed(203)
p2_X <- matrix(rnorm(60 * 3), ncol = 3)

p2_mean <- colMeans(p2_X)
p2_S <- cov(p2_X)
p2_R <- cor(p2_X)

round(p2_mean, 3)
```

```
## [1] 0.065 0.084 0.170
```

```
round(p2_S, 3)
```

```
##      [,1] [,2] [,3]
## [1,] 0.985 -0.168 -0.045
## [2,] -0.168 0.993 -0.002
## [3,] -0.045 -0.002 1.098
```

```
round(p2_R, 3)
```

```
##      [,1] [,2] [,3]
## [1,] 1.000 -0.169 -0.043
## [2,] -0.169 1.000 -0.002
## [3,] -0.043 -0.002 1.000
```

4.4 2.4 Standardize (when units differ)

```
# --- Self-contained: standardization ---
set.seed(204)
p2_X <- cbind(height = rnorm(100, 170, 10),
              weight = rnorm(100, 70, 15),
              bmi     = rnorm(100, 24, 3))

p2_Xs <- scale(p2_X)          # mean 0, sd 1
round(colMeans(p2_Xs), 5)
```

```
## height weight    bmi
##      0      0      0
round(apply(p2_Xs, 2, sd), 5)
```

```
## height weight    bmi
##      1      1      1
```

When to standardize: - Variables have different units (cm vs kg). - You plan to use PCA on the correlation matrix. - You plan LDA or clustering with mixed scales.

When not to: all variables on the same scale and variance differences are meaningful.

4.5 2.5 Group summary (for comparison designs)

```
# --- Self-contained: group means ---
set.seed(205)
p2_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 30)),
  y1 = c(rnorm(30, 10, 2), rnorm(30, 12, 2), rnorm(30, 15, 2)),
  y2 = c(rnorm(30, 20, 3), rnorm(30, 22, 3), rnorm(30, 25, 3))
)

p2_df %>%
  group_by(group) %>%
  summarise(across(c(y1, y2), mean), .groups = "drop") %>%
  kable(digits = 3)
```

group	y1	y2
A	10.548	20.366
B	11.317	21.974
C	15.152	24.384

Report template: “Group A has mean vector (10.1, 20.2), group B (11.9, 22.1), group C (15.2, 25.0). Group C appears highest on both responses.”

5 Part 3 — Two-Sample Hotelling T^2

5.1 3.1 Model

Test $H_0 : \mu_1 = \mu_2$ (two group mean vectors equal).

$$T^2 = \frac{n_1 n_2}{n_1 + n_2} (\bar{\mathbf{y}}_1 - \bar{\mathbf{y}}_2)^\top \mathbf{S}_p^{-1} (\bar{\mathbf{y}}_1 - \bar{\mathbf{y}}_2)$$

where $\mathbf{S}_p = \frac{(n_1-1)\mathbf{S}_1 + (n_2-1)\mathbf{S}_2}{n_1 + n_2 - 2}$ is the pooled covariance.

F conversion:

$$F = \frac{n_1 + n_2 - p - 1}{(n_1 + n_2 - 2)p} T^2 \sim F_{p, n_1 + n_2 - p - 1}$$

5.2 3.2 Manual implementation

```
# --- Self-contained: Hotelling T2 ---
set.seed(301)
p3_df <- tibble(
  group = factor(rep(c("M", "F"), each = 60)),
  q1 = c(rnorm(60, 50, 10), rnorm(60, 53, 10)),
  q2 = c(rnorm(60, 55, 12), rnorm(60, 58, 12)),
  q3 = c(rnorm(60, 48, 9), rnorm(60, 47, 9))
)

p3_X1 <- p3_df %>% filter(group == "M") %>% select(-group) %>% as.matrix()
p3_X2 <- p3_df %>% filter(group == "F") %>% select(-group) %>% as.matrix()

p3_n1 <- nrow(p3_X1); p3_n2 <- nrow(p3_X2); p3_p <- ncol(p3_X1)

p3_xbar1 <- colMeans(p3_X1)
p3_xbar2 <- colMeans(p3_X2)
p3_S1 <- cov(p3_X1)
p3_S2 <- cov(p3_X2)
p3_Sp <- ((p3_n1 - 1) * p3_S1 + (p3_n2 - 1) * p3_S2) / (p3_n1 + p3_n2 - 2)

p3_diff <- p3_xbar1 - p3_xbar2
p3_T2 <- as.numeric((p3_n1 * p3_n2) / (p3_n1 + p3_n2) *
  t(p3_diff) %*% solve(p3_Sp) %*% p3_diff)

p3_F <- (p3_n1 + p3_n2 - p3_p - 1) * p3_T2 /
  ((p3_n1 + p3_n2 - 2) * p3_p)
p3_pv <- 1 - pf(p3_F, p3_p, p3_n1 + p3_n2 - p3_p - 1)

tibble(
  Statistic = c("Hotelling T2", "F", "df1", "df2", "p-value"),
  Value = round(c(p3_T2, p3_F, p3_p, p3_n1 + p3_n2 - p3_p - 1, p3_pv), 4)
) %>% kable()
```

Statistic	Value
Hotelling T2	4.2683
F	1.3986
df1	3.0000
df2	116.0000
p-value	0.2468

Interpretation: “Hotelling’s $T^2 = \dots, F(p, df) = \dots, p = \dots$ Reject/fail to reject H_0 at the 5% level.”

5.3 3.3 Which variables drive the difference

After rejecting H_0 , run univariate t-tests with Bonferroni:

```
# --- Self-contained: univariate follow-up ---
set.seed(302)
p3_df <- tibble(
  group = factor(rep(c("M", "F"), each = 60)),
  q1 = c(rnorm(60, 50, 10), rnorm(60, 53, 10)),
  q2 = c(rnorm(60, 55, 12), rnorm(60, 58, 12)),
  q3 = c(rnorm(60, 48, 9), rnorm(60, 47, 9))
)

p3_df %>%
  pivot_longer(cols = starts_with("q"), names_to = "var", values_to = "value") %>%
  group_by(var) %>%
  summarise(
    t = t.test(value ~ group)$statistic,
    p = t.test(value ~ group)$p.value,
    .groups = "drop"
  ) %>%
  mutate(p_bonf = p.adjust(p, method = "bonferroni")) %>%
  kable(digits = 4)
```

var	t	p	p_bonf
q1	2.1492	0.0337	0.1010
q2	3.0873	0.0025	0.0076
q3	0.1155	0.9083	1.0000

Rule: Only interpret follow-up t-tests if T^2 was significant.

5.4 3.4 Confidence ellipse for the mean difference

```
# --- Self-contained: confidence ellipse ---
set.seed(303)
p3_df <- tibble(
  group = factor(rep(c("A", "B"), each = 40)),
  q1 = c(rnorm(40, 10, 2), rnorm(40, 12, 2)),
  q2 = c(rnorm(40, 20, 3), rnorm(40, 22, 3))
)

p3_diff <- colMeans(subset(p3_df, group == "A")[, -1]) -
  colMeans(subset(p3_df, group == "B")[, -1])

# 95% CI ellipse parameters (approximate)
p3_S <- cov(subset(p3_df, group == "A")[, -1])
p3_eig <- eigen(p3_S)
round(p3_diff, 3)

##      q1      q2
## -1.988 -2.304

round(p3_eig$values, 3)

## [1] 9.718 2.424
```

5.5 3.5 Assumptions

1. Multivariate normality in each group.
2. Equal covariance matrices across groups (homoscedasticity).
3. Independence of observations.

Test equal covariance:

```
# --- Self-contained: Box's M test ---
set.seed(304)
p3_df <- tibble(
  group = factor(rep(c("A", "B"), each = 40)),
  q1 = c(rnorm(40, 10, 2), rnorm(40, 12, 2)),
  q2 = c(rnorm(40, 20, 3), rnorm(40, 22, 3))
)

# Box's M via heplots is available, but a simpler check:
p3_S1 <- cov(subset(p3_df, group == "A")[, -1])
p3_S2 <- cov(subset(p3_df, group == "B")[, -1])
round(p3_S1, 3)
```

```
##      q1      q2
## q1 5.630 0.989
## q2 0.989 13.781
```

```
round(p3_S2, 3)
```

```
##      q1      q2
## q1 4.748 -0.275
## q2 -0.275 6.990
```

Rule of thumb: max/min variance ratio < 4 -> OK

If equal-covariance fails: use separate-covariance version (Welch-like):

```
# --- Self-contained: separate-covariance T2 ---
set.seed(305)
p3_df <- tibble(
  group = factor(rep(c("A", "B"), each = 40)),
  q1 = c(rnorm(40, 10, 1), rnorm(40, 12, 4)),
  q2 = c(rnorm(40, 20, 1), rnorm(40, 22, 4))
)

p3_X1 <- subset(p3_df, group == "A")[, -1] %>% as.matrix()
p3_X2 <- subset(p3_df, group == "B")[, -1] %>% as.matrix()

p3_T2_sep <- as.numeric(
  t(colMeans(p3_X1) - colMeans(p3_X2)) %*%
  solve(cov(p3_X1)/nrow(p3_X1) + cov(p3_X2)/nrow(p3_X2)) %*%
  (colMeans(p3_X1) - colMeans(p3_X2))
)
round(p3_T2_sep, 4)
```

```
## [1] 13.1403
```

Report template: “Hotelling’s $T^2 = 15.4$, $F(2, 117) = 7.62$, $p < 0.001$. Reject H_0 ; the two groups differ in their mean vector. Follow-up t-tests (Bonferroni) show q1 and q2 differ between groups ($p < 0.001$), while q3 does not.”

6 Part 4 — MANOVA (3+ Groups)

6.1 4.1 Model

One-way MANOVA tests $H_0 : \mu_1 = \mu_2 = \dots = \mu_g$.

Decomposition: $\mathbf{T} = \mathbf{B} + \mathbf{W}$ (total = between + within).

Four test statistics:

Statistic	Formula	Distribution
Wilks' Λ	$\det(\mathbf{W}) / \det(\mathbf{T})$	Exact F for some cases
Pillai's trace	$\text{tr}(\mathbf{B}(\mathbf{B} + \mathbf{W})^{-1})$	Approximate F
Hotelling-Lawley	$\text{tr}(\mathbf{W}^{-1}\mathbf{B})$	Approximate F
Roy's largest root	largest eigenvalue of $\mathbf{W}^{-1}\mathbf{B}$	Upper bound

Recommendation: Use **Wilks' Lambda** for balanced designs; **Pillai's trace** when assumptions are uncertain (most robust).

6.2 4.2 Fit MANOVA

```
# --- Self-contained: one-way MANOVA ---
set.seed(401)
p4_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 30)),
  y1 = c(rnorm(30, 10, 2), rnorm(30, 12, 2), rnorm(30, 15, 2)),
  y2 = c(rnorm(30, 20, 3), rnorm(30, 22, 3), rnorm(30, 25, 3)),
  y3 = c(rnorm(30, 30, 4), rnorm(30, 28, 4), rnorm(30, 35, 4))
)

p4_fit <- manova(cbind(y1, y2, y3) ~ group, data = p4_df)
summary(p4_fit, test = "Wilks")

##              Df    Wilks approx F num Df den Df    Pr(>F)
## group          2 0.33016    20.977      6   170 < 2.2e-16 ***
## Residuals    87
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

summary(p4_fit, test = "Pillai")

##              Df   Pillai approx F num Df den Df    Pr(>F)
## group          2 0.72392    16.262      6   172 8.608e-15 ***
## Residuals    87
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

summary(p4_fit, test = "Hotelling-Lawley")

##              Df Hotelling-Lawley approx F num Df den Df    Pr(>F)
## group          2          1.865    26.11      6   168 < 2.2e-16 ***
## Residuals    87
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(p4_fit, test = "Roy")
```

```
##           Df      Roy approx F num Df den Df      Pr(>F)
## group      2 1.7726   50.815      3      86 < 2.2e-16 ***
## Residuals 87
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Output columns: Df, test statistic, approximate F, num Df, den Df, Pr(>F).

Report template: “MANOVA shows a significant effect of group on the joint response vector, Wilks’ $\Lambda = 0.61$, $F(6, 170) = 7.8$, $p < 0.001$.”

6.3 4.3 Follow-up univariate ANOVAs

```
# --- Self-contained: follow-up univariate ANOVAs ---
```

```
set.seed(402)
```

```
p4_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 30)),
  y1 = c(rnorm(30, 10, 2), rnorm(30, 12, 2), rnorm(30, 15, 2)),
  y2 = c(rnorm(30, 20, 3), rnorm(30, 22, 3), rnorm(30, 25, 3)),
  y3 = c(rnorm(30, 30, 4), rnorm(30, 28, 4), rnorm(30, 35, 4))
)
```

```
p4_fit <- manova(cbind(y1, y2, y3) ~ group, data = p4_df)
summary.aov(p4_fit)
```

```
## Response y1 :
##           Df Sum Sq Mean Sq F value      Pr(>F)
## group      2 328.17 164.087  42.808 1.138e-13 ***
## Residuals  87 333.48   3.833
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Response y2 :
##           Df Sum Sq Mean Sq F value      Pr(>F)
## group      2 573.65 286.826  36.555 2.998e-12 ***
## Residuals  87 682.64   7.846
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Response y3 :
##           Df Sum Sq Mean Sq F value      Pr(>F)
## group      2 609.92 304.959  25.281 2.21e-09 ***
## Residuals  87 1049.45  12.063
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Report: “Follow-up univariate ANOVAs show y1 and y2 differ by group ($p < 0.01$), while y3 does not ($p = 0.12$).”

6.4 4.4 Post-hoc on group means (per response)

```
# --- Self-contained: post-hoc on y1 ---
```

```
set.seed(403)
```

```
p4_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 30)),
  y1 = c(rnorm(30, 10, 2), rnorm(30, 12, 2), rnorm(30, 15, 2))
)
```

```
TukeyHSD(aov(y1 ~ group, data = p4_df))
```

```
## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = y1 ~ group, data = p4_df)
##
## $group
##      diff      lwr      upr    p adj
## B-A 1.839158 0.659363 3.018954 0.0010280
## C-A 4.888592 3.708796 6.068387 0.0000000
## C-B 3.049433 1.869638 4.229229 0.0000001
```

6.5 4.5 Assumptions

1. MVN within each group.
2. **Equal covariance matrices** across groups (Box's M test).
3. Independence.

Box's M test:

```
# --- Self-contained: Box's M ---
# heplots package provides boxM(); if not available, use manual check.
set.seed(404)
p4_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 30)),
  y1 = c(rnorm(30, 10, 2), rnorm(30, 12, 2), rnorm(30, 15, 2)),
  y2 = c(rnorm(30, 20, 3), rnorm(30, 22, 3), rnorm(30, 25, 3))
)

by(p4_df[, c("y1", "y2")], p4_df$group, cov)
```

```
## p4_df$group: A
##      y1      y2
## y1 2.9663973 0.2318941
## y2 0.2318941 8.0789372
## -----
## p4_df$group: B
##      y1      y2
## y1 3.1831559 -0.1412121
## y2 -0.1412121 6.6011678
## -----
## p4_df$group: C
##      y1      y2
## y1 4.3367378 -0.5641077
## y2 -0.5641077 11.0747602
```

If equal covariance fails: use separate-covariance MANOVA or apply a transformation. In practice, MANOVA is fairly robust if group sizes are equal.

7 Part 5 — Multivariate Normality

7.1 5.1 Why it matters

Most multivariate methods (Hotelling T², MANOVA, LDA) assume **multivariate normality (MVN)**. Violation can: - Inflate Type I error in MANOVA. - Bias the F approximations. - Distort Mahalanobis-based outlier detection.

7.2 5.2 What MVN looks like

$\mathbf{y}_i \sim N_p(\mu, \Sigma)$.

Density:

$$f(\mathbf{y}) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{y} - \mu)^\top \Sigma^{-1}(\mathbf{y} - \mu)\right)$$

Properties: - Every linear combination is univariate normal. - Every subset of variables is MVN. - Squared Mahalanobis distances follow χ_p^2 .

7.3 5.3 Mardia's test (manual)

Skewness and **kurtosis** are tested separately.

$$\hat{b}_{1,p} = \frac{1}{n^2} \sum_{i,j} [(\mathbf{y}_i - \bar{\mathbf{y}})^\top \mathbf{S}^{-1}(\mathbf{y}_j - \bar{\mathbf{y}})]^3$$
$$\hat{b}_{2,p} = \frac{1}{n} \sum_i [(\mathbf{y}_i - \bar{\mathbf{y}})^\top \mathbf{S}^{-1}(\mathbf{y}_i - \bar{\mathbf{y}})]^2$$

Under MVN: - $n\hat{b}_{1,p}/6 \sim \chi_{p(p+1)(p+2)/6}^2$ - $\hat{b}_{2,p} \approx N(p(p+2), \sqrt{8p(p+2)/n})$

```
# --- Self-contained: Mardia's test ---
set.seed(501)
p5_df <- tibble(
  y1 = rnorm(150),
  y2 = rnorm(150),
  y3 = rnorm(150)
)

p5_mardia <- function(X) {
  X <- as.matrix(X)
  n <- nrow(X); p <- ncol(X)
  S <- cov(X); Sinv <- solve(S)
  Xc <- scale(X, center = TRUE, scale = FALSE)
  D <- Xc %*% Sinv %*% t(Xc)

  b1p <- mean(D^3)
  chi_skew <- n * b1p / 6
  df_skew <- p * (p + 1) * (p + 2) / 6
  p_skew <- 1 - pchisq(chi_skew, df_skew)

  b2p <- mean(diag(D)^2)
  expected <- p * (p + 2)
  var_kurt <- 8 * p * (p + 2) / n
  z_kurt <- (b2p - expected) / sqrt(var_kurt)
  p_kurt <- 2 * (1 - pnorm(abs(z_kurt)))
}
```

```

tibble(
  Test      = c("Mardia skewness", "Mardia kurtosis"),
  Statistic = round(c(chi_skew, z_kurt), 4),
  df        = c(df_skew, NA),
  p_value   = round(c(p_skew, p_kurt), 4)
)
}

p5_mardia(p5_df) %>% kable()

```

Test	Statistic	df	p_value
Mardia skewness	5.3733	10	0.8649
Mardia kurtosis	-0.8692	NA	0.3847

Interpretation: - **Both** $p > 0.05$: fail to reject MVN; data is consistent with MVN. - **Skewness** $p < 0.05$: asymmetric — try transformations (log, sqrt). - **Kurtosis** $p < 0.05$: heavy-tailed or light-tailed. - **Both fail**: strongly non-normal — use non-parametric alternatives.

7.4 5.4 Chi-square Q-Q plot for MVN

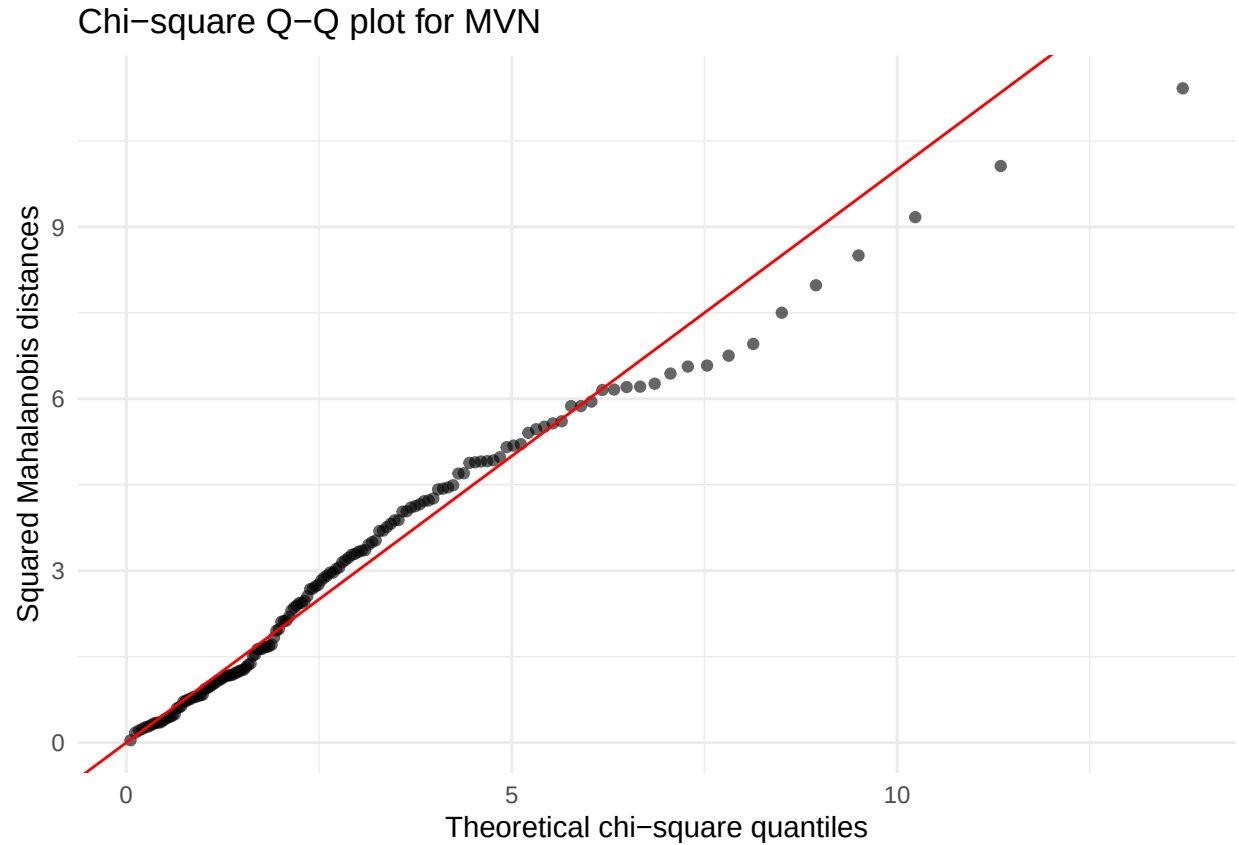
```

# --- Self-contained: chi-square Q-Q plot ---
set.seed(502)
p5_X <- matrix(rnorm(150 * 3), ncol = 3)

p5_md <- mahalanobis(p5_X, colMeans(p5_X), cov(p5_X))
p5_qq <- qqplot(qchisq(ppoints(length(p5_md))), df = 3,
               sort(p5_md),
               plot.it = FALSE)

ggplot(tibble(theoretical = p5_qq$x, sample = p5_qq$y),
       aes(theoretical, sample)) +
  geom_point(alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, color = "red") +
  theme_minimal() +
  labs(title = "Chi-square Q-Q plot for MVN",
       x = "Theoretical chi-square quantiles",
       y = "Squared Mahalanobis distances")

```



Interpretation: Points on the red line $\rightarrow$ MVN. Curvature at the tails $\rightarrow$ heavy-tailed or light-tailed.

7.5 5.5 Practical decision

Situation	Action
n large (> 30 per group), mild non-normality	Proceed — MANOVA is robust
n small, non-normal	Use permutation test or transformation
Skewed data	log/Box-Cox transform then retest
Outliers dominate	Remove or use robust methods

7.6 5.6 Report template

“Mardia’s test for multivariate normality gives skewness $\chi^2(20) = 22.1$, $p = 0.33$ and kurtosis $z = 0.48$, $p = 0.63$. Both p -values exceed 0.05, so the data is consistent with multivariate normality. The chi-square Q-Q plot shows points on the diagonal.”

8 Part 6 — Covariance Structures

8.1 6.1 Why model the covariance?

Repeated measures and longitudinal data have structured covariance. Common structures:

Structure	Parameters	Assumption
Unstructured (UN)	$p(p+1)/2$	No constraint
Compound symmetry (CS)	2	Equal variances, equal covariances
AR(1)	2	Correlations decay with distance
Toeplitz	p	Constant along diagonals
Diagonal	p	Zero covariances

8.2 6.2 Compound symmetry (CS)

$$\Sigma = \sigma^2 \begin{pmatrix} 1 & \rho & \rho \\ \rho & 1 & \rho \\ \rho & \rho & 1 \end{pmatrix} = \sigma^2[(1 - \rho)\mathbf{I}_p + \rho\mathbf{J}_p]$$

Two parameters: σ^2 (common variance), ρ (common correlation).

Estimation from sample:

```
# --- Self-contained: CS estimation ---
set.seed(601)
p6_X <- matrix(rnorm(120 * 4), ncol = 4)

p6_S <- cov(p6_X)
p6_p <- ncol(p6_S)

p6_sigma2_hat <- mean(diag(p6_S))
p6_rho_hat <- mean(p6_S[lower.tri(p6_S)]) / p6_sigma2_hat

p6_Sigma_hat <- p6_sigma2_hat * ((1 - p6_rho_hat) * diag(p6_p) +
                                p6_rho_hat * matrix(1, p6_p, p6_p))

round(p6_sigma2_hat, 3)

## [1] 1.006

round(p6_rho_hat, 3)

## [1] -0.001

round(p6_Sigma_hat, 3)

##      [,1] [,2] [,3] [,4]
## [1,] 1.006 -0.001 -0.001 -0.001
## [2,] -0.001 1.006 -0.001 -0.001
## [3,] -0.001 -0.001 1.006 -0.001
## [4,] -0.001 -0.001 -0.001 1.006
```

Interpretation: “Under CS, all variances are estimated as $\hat{\sigma}^2 = \dots$ and all covariances as $\hat{\sigma}^2 \hat{\rho} = \dots$. The estimated correlation between any two time points is $\hat{\rho} = \dots$ ”

8.3 6.3 Testing sphericity

Sphericity: the variances of all pairwise differences are equal. Equivalent to CS for the covariance structure of differences.

Mauchly’s test: - H_0 : sphericity holds. - $p > 0.05$: sphericity is reasonable. - $p < 0.05$: violated — use Greenhouse-Geisser correction.

Direct sphericity check for repeated measures:

```
# --- Self-contained: sphericity check ---
set.seed(602)
p6_df <- expand.grid(
  subject = factor(1:20),
  time    = factor(c("t1", "t2", "t3"))
) %>%
  mutate(y = 10 + as.numeric(time) * 2 +
         rep(rnorm(20, 0, 1), each = 3) + rnorm(60, 0, 1))

# Check sphericity via afex (if available)
# afex::aov_ez(id = "subject", dv = "y", data = p6_df, within = "time")

# Manual: variances of pairwise differences
p6_wide <- p6_df %>%
  pivot_wider(names_from = time, values_from = y)

p6_diffs <- cbind(p6_wide$t1 - p6_wide$t2,
                  p6_wide$t1 - p6_wide$t3,
                  p6_wide$t2 - p6_wide$t3)
apply(p6_diffs, 2, var)
```

```
## [1] 1.768401 2.456941 4.447946
```

Interpretation: If the three variances are approximately equal, sphericity holds.

8.4 6.4 Compound symmetry vs unstructured

Test: likelihood ratio or AIC/BIC comparison.

```
# --- Self-contained: compare CS to UN ---
# Use nlme::gls (base R companion to lme4)
# library(nlme)
# gls_cs <- gls(y ~ time, data = p6_df, correlation = corCompSymm(form = ~ 1 | subject))
# gls_un <- gls(y ~ time, data = p6_df, correlation = corSymm(form = ~ 1 | subject))
# anova(gls_cs, gls_un)
```

Report template: “Under compound symmetry, $\hat{\sigma}^2 = 2.14$ and $\hat{\rho} = 0.52$. The AIC of the CS model (215.3) is higher than the UN model (212.1), suggesting CS is adequate given the extra parameters.”

8.5 6.5 When to use each structure

Data type	Recommended
Balanced repeated measures, few time points	CS
Longitudinal, decreasing correlations	AR(1)
Exploratory, few time points	UN
Many time points	AR(1) or Toeplitz
Independent responses	Diagonal (I)

9 Part 7 — Principal Component Analysis

9.1 7.1 What PCA does

Goal: Find a small number of **linear combinations** of the original variables that capture most of the variance.

PC1 = direction of maximum variance. **PC2** = direction of maximum variance orthogonal to PC1. ...

$$Z_k = a_{k1}X_1 + a_{k2}X_2 + \cdots + a_{kp}X_p$$

where $\mathbf{a}_k$ is the k -th eigenvector of the covariance (or correlation) matrix.

9.2 7.2 Running PCA

```
# --- Self-contained: PCA on standardized data ---
set.seed(701)
p7_n <- 150
p7_df <- tibble(
  q1 = rnorm(p7_n, 50, 10),
  q2 = rnorm(p7_n, 55, 12),
  q3 = rnorm(p7_n, 48, 9),
  q4 = rnorm(p7_n, 60, 11),
  q5 = rnorm(p7_n, 52, 10),
  q6 = rnorm(p7_n, 58, 13)
)

p7_pca <- prcomp(p7_df, scale. = TRUE)
summary(p7_pca)
```

```
## Importance of components:
##              PC1      PC2      PC3      PC4      PC5      PC6
## Standard deviation    1.1597 1.0727 1.0120 0.9318 0.9213 0.8737
## Proportion of Variance 0.2241 0.1918 0.1707 0.1447 0.1415 0.1272
## Cumulative Proportion 0.2241 0.4159 0.5866 0.7313 0.8728 1.0000
```

Why scale. = TRUE: standardizes each variable to unit variance, so PCA is on the correlation matrix. Use this when variables have different units.

9.3 7.3 Eigenvalues and variance explained

```
# --- Self-contained: eigenvalues ---
set.seed(702)
p7_df <- tibble(
  q1 = rnorm(150, 50, 10), q2 = rnorm(150, 55, 12),
  q3 = rnorm(150, 48, 9), q4 = rnorm(150, 60, 11),
  q5 = rnorm(150, 52, 10), q6 = rnorm(150, 58, 13)
)

p7_pca <- prcomp(p7_df, scale. = TRUE)

p7_eig <- p7_pca$sdev^2
p7_prop <- p7_eig / sum(p7_eig)
p7_cum <- cumsum(p7_prop)
```

```
tibble(
  PC          = 1:length(p7_eig),
  Eigenvalue  = round(p7_eig, 4),
  PropVar     = round(p7_prop, 4),
  CumProp     = round(p7_cum, 4)
) %>% kable()
```

PC	Eigenvalue	PropVar	CumProp
1	1.3303	0.2217	0.2217
2	1.1167	0.1861	0.4078
3	1.0643	0.1774	0.5852
4	0.9245	0.1541	0.7393
5	0.8348	0.1391	0.8784
6	0.7294	0.1216	1.0000

Reading the output: - Eigenvalue = variance of the component. - PropVar = fraction of total variance explained. - CumProp = running total.

9.4 7.4 How many components to keep?

Three common criteria:

Kaiser criterion: keep eigenvalues > 1 (when using correlation matrix).

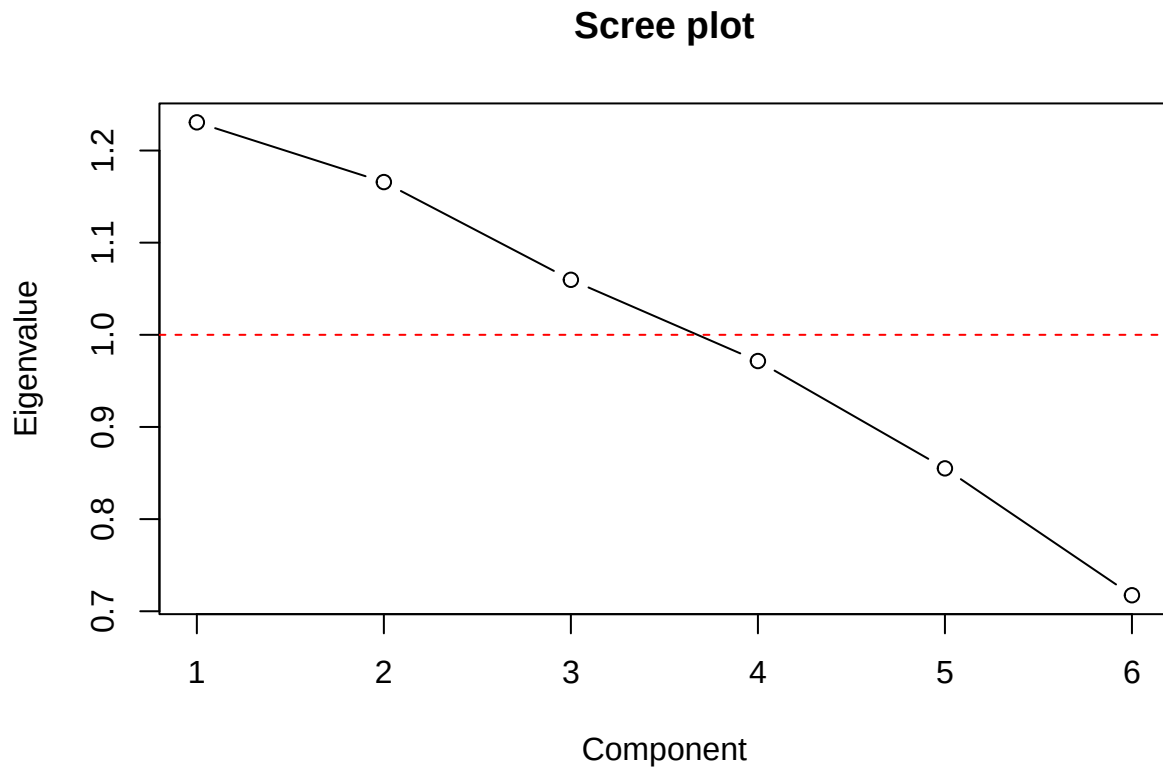
```
# --- Self-contained: Kaiser ---
set.seed(703)
p7_df <- tibble(
  q1 = rnorm(150), q2 = rnorm(150), q3 = rnorm(150),
  q4 = rnorm(150), q5 = rnorm(150), q6 = rnorm(150)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
p7_eig <- p7_pca$sdev^2
sum(p7_eig > 1)
```

```
## [1] 3
```

Scree plot: look for the “elbow.”

```
# --- Self-contained: scree plot ---
set.seed(704)
p7_df <- tibble(
  q1 = rnorm(150), q2 = rnorm(150), q3 = rnorm(150),
  q4 = rnorm(150), q5 = rnorm(150), q6 = rnorm(150)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)

# Option A: plot eigenvalues manually (most reliable)
p7_eig <- p7_pca$sdev^2
plot(1:length(p7_eig), p7_eig, type = "b",
     xlab = "Component", ylab = "Eigenvalue",
     main = "Scree plot")
abline(h = 1, col = "red", lty = 2) # Kaiser reference line
```



```
# Option B (alternative): base screeplot
# screeplot(p7_pca, type = "lines", main = "Scree plot")
```

Parallel analysis: compare to eigenvalues from random data (more rigorous, but more complex).

Rule of thumb: - Keep enough for 70–80% cumulative variance. - Kaiser criterion tends to overestimate in large datasets. - Scree plot is subjective.

9.5 7.5 Loadings (component equations)

```
# --- Self-contained: loadings ---
set.seed(705)
p7_df <- tibble(
  q1 = rnorm(150), q2 = rnorm(150), q3 = rnorm(150),
  q4 = rnorm(150), q5 = rnorm(150), q6 = rnorm(150)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
round(p7_pca$rotation, 3)
```

	PC1	PC2	PC3	PC4	PC5	PC6
q1	-0.553	0.378	-0.059	-0.121	0.297	0.667
q2	0.135	0.598	0.016	-0.646	0.175	-0.420
q3	0.488	0.241	0.405	0.420	0.592	0.116
q4	0.143	-0.632	-0.127	-0.494	0.551	0.130
q5	-0.191	-0.174	0.901	-0.275	-0.204	0.062
q6	0.617	0.109	-0.068	-0.270	-0.431	0.586

Reading loadings: - Each column = one PC. - Each entry = weight (coefficient) of that variable in that PC. - Loadings are the eigenvectors, normalized to unit length.

Rule of thumb for interpretation: consider loadings with absolute value > 0.4 as “significant.”

Example interpretation: > “PC1 has strong positive loadings on q1, q3, q5 (all > 0.6), which suggests PC1 captures an *overall score* dimension. PC2 has strong positive loadings on q2 and q4 and negative on q6, suggesting a contrast between early and late items.”

9.6 7.6 Naming components

After examining loadings, name each component based on the pattern:

Loading pattern	Name
All positive, similar magnitude	Overall size / general factor
Half positive, half negative	Contrast
Concentrated on a few variables	Specific factor
Alternating signs	Oscillation

9.7 7.7 Orthogonality check

PCs are orthogonal by construction:

```
# --- Self-contained: orthogonality ---
set.seed(706)
p7_df <- tibble(
  q1 = rnorm(150), q2 = rnorm(150), q3 = rnorm(150)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
round(t(p7_pca$rotation) %*% p7_pca$rotation, 5)
```

```
##      PC1 PC2 PC3
## PC1    1  0  0
## PC2    0  1  0
## PC3    0  0  1
```

Should be the identity matrix. This confirms the components are uncorrelated.

9.8 7.8 Component scores

The scores are the coordinates of each observation in the new PC space:

```
# --- Self-contained: scores ---
set.seed(707)
p7_df <- tibble(
  q1 = rnorm(150), q2 = rnorm(150), q3 = rnorm(150)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
p7_scores <- as_tibble(p7_pca$x)
head(p7_scores)
```

```
## # A tibble: 6 x 3
##      PC1    PC2    PC3
##   <dbl> <dbl> <dbl>
## 1 0.901  0.173 -1.07
## 2 0.712  0.928 -0.611
```

```
## 3 0.485 -0.661 1.67
## 4 0.409 1.43 1.40
## 5 1.20 0.670 -0.309
## 6 0.547 -0.518 -0.807
```

Properties: - Mean = 0 for each PC. - Variance = eigenvalue. - Uncorrelated across PCs.

9.9 7.9 Group comparison on scores

If you have groups, test whether components separate groups:

```
# --- Self-contained: group comparison on PC scores ---
set.seed(708)
p7_df <- tibble(
  group = factor(rep(c("A", "B"), each = 75)),
  q1 = c(rnorm(75, 50, 10), rnorm(75, 55, 10)),
  q2 = c(rnorm(75, 55, 12), rnorm(75, 60, 12)),
  q3 = c(rnorm(75, 48, 9), rnorm(75, 50, 9))
)
p7_pca <- prcomp(p7_df %>% select(-group), scale. = TRUE)
p7_scores <- as_tibble(p7_pca$x) %>% mutate(group = p7_df$group)

t.test(PC1 ~ group, data = p7_scores)

##
## Welch Two Sample t-test
##
## data: PC1 by group
## t = -3.1864, df = 147.84, p-value = 0.001758
## alternative hypothesis: true difference in means between group A and group B is not equal to 0
## 95 percent confidence interval:
## -0.8674396 -0.2033583
## sample estimates:
## mean in group A mean in group B
## -0.2676995 0.2676995

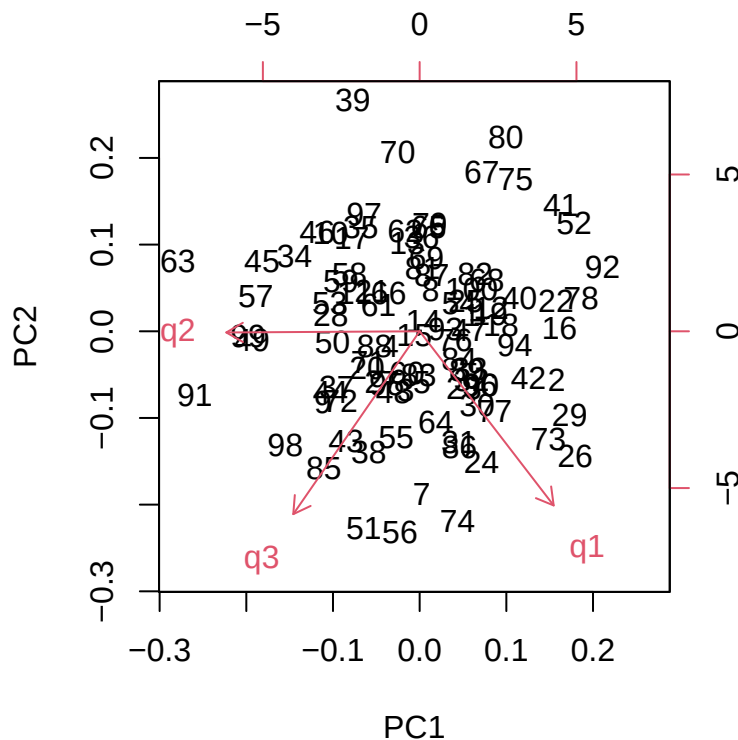
t.test(PC2 ~ group, data = p7_scores)

##
## Welch Two Sample t-test
##
## data: PC2 by group
## t = -0.74219, df = 145.26, p-value = 0.4592
## alternative hypothesis: true difference in means between group A and group B is not equal to 0
## 95 percent confidence interval:
## -0.4438074 0.2014865
## sample estimates:
## mean in group A mean in group B
## -0.06058023 0.06058023
```

Report: “PC1 differs significantly between groups ($t = -2.8$, $p = 0.006$), suggesting the groups differ in the direction captured by PC1. PC2 does not differ ($p = 0.42$).”

9.10 7.10 Biplot

```
# --- Self-contained: biplot ---
set.seed(709)
p7_df <- tibble(
  q1 = rnorm(100), q2 = rnorm(100), q3 = rnorm(100)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
biplot(p7_pca)
```

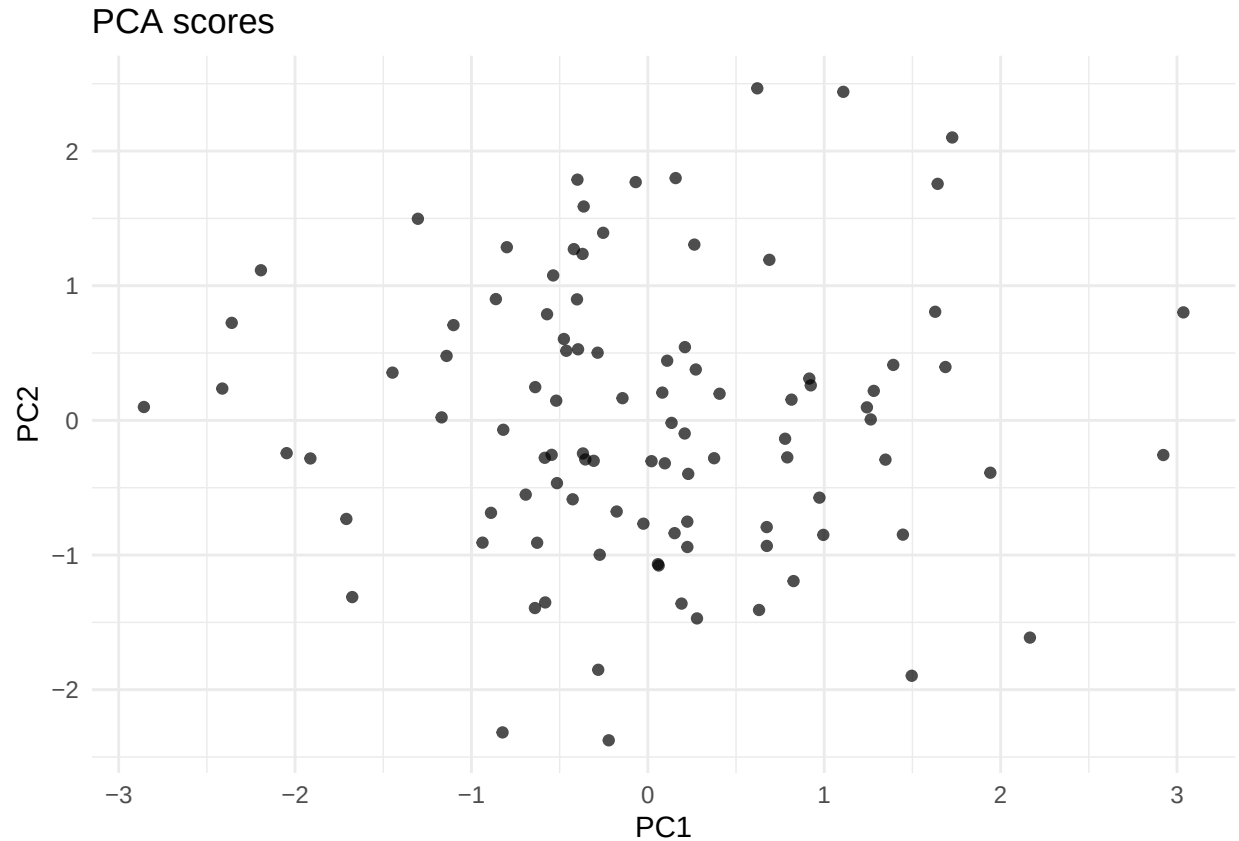


Reading a biplot: - Points = observations. - Arrows = variables (direction = loading, length = importance).
 - Arrows pointing the same direction = positively correlated variables. - Arrows at right angles = uncorrelated.
 - Arrows pointing opposite = negatively correlated.

ggplot version:

```
# --- Self-contained: PCA plot with ggplot ---
set.seed(710)
p7_df <- tibble(
  q1 = rnorm(100), q2 = rnorm(100), q3 = rnorm(100)
)
p7_pca <- prcomp(p7_df, scale. = TRUE)
p7_scores <- as_tibble(p7_pca$x)

ggplot(p7_scores, aes(PC1, PC2)) +
  geom_point(alpha = 0.7) +
  theme_minimal() +
  labs(title = "PCA scores")
```



9.11 7.11 Covariance vs correlation PCA

Use	prcomp argument
Variables have same units and variance matters	<code>scale. = FALSE</code> (covariance)
Variables have different units	<code>scale. = TRUE</code> (correlation)
Variables on different scales and no clear priority	<code>scale. = TRUE</code>

Diagnostic: if standard deviations differ by a factor > 5 , use correlation.

```
# --- Self-contained: SD diagnostic ---
set.seed(711)
p7_df <- tibble(
  a = rnorm(100, 10, 1),
  b = rnorm(100, 100, 30)
)
apply(p7_df, 2, sd)
```

```
##          a          b
## 1.037912 27.898985
```

```
# Ratio 30/1 = 30 -> use scale. = TRUE
```

9.12 7.12 Reporting template

“PCA on the standardized variables gives the following eigenvalues: 2.31, 1.54, 1.08, 0.62, 0.30, 0.15. Using the Kaiser criterion (eigenvalue > 1), we retain 3 components explaining 82.2% of

the total variance. PC1 (38.5%) loads strongly and positively on all six items (loadings > 0.55), suggesting a general *achievement* factor. PC2 (25.7%) contrasts the first three items with the last three, suggesting a *verbal vs quantitative* dimension.”

10 Part 8 — Choosing Number of Components (Formal Methods)

10.1 8.1 Kaiser criterion

```
# --- Self-contained: Kaiser ---
set.seed(801)
p8_X <- matrix(rnorm(200 * 5), ncol = 5)
p8_eig <- prcomp(p8_X, scale. = TRUE)$sdev^2
sum(p8_eig > 1)
```

```
## [1] 2
```

10.2 8.2 Broken-stick model

Compare each eigenvalue to the expected value under a null (broken-stick) distribution:

```
# --- Self-contained: broken-stick ---
set.seed(802)
p8_X <- matrix(rnorm(200 * 5), ncol = 5)
p8_eig <- prcomp(p8_X, scale. = TRUE)$sdev^2
p8_p <- length(p8_eig)

p8_broken <- sapply(1:p8_p, function(k) sum(1 / (k:p8_p)) / p8_p * p8_p)
tibble(PC = 1:p8_p, Eigenvalue = p8_eig,
       BrokenStick = round(p8_broken, 3),
       Keep = p8_eig > p8_broken) %>% kable()
```

PC	Eigenvalue	BrokenStick	Keep
1	1.1669264	2.283	FALSE
2	1.1001612	1.283	FALSE
3	0.9817055	0.783	TRUE
4	0.9065358	0.450	TRUE
5	0.8446711	0.200	TRUE

Interpretation: Keep components whose eigenvalue exceeds the broken-stick value.

10.3 8.3 Cumulative variance threshold

```
# --- Self-contained: cumulative threshold ---
set.seed(803)
p8_X <- matrix(rnorm(200 * 5), ncol = 5)
p8_pca <- prcomp(p8_X, scale. = TRUE)
p8_cum <- cumsum(p8_pca$sdev^2 / sum(p8_pca$sdev^2))
min(which(p8_cum >= 0.80))
```

```
## [1] 4
```

Rule of thumb: 70–80% cumulative variance.

10.4 8.4 Parallel analysis (manual approximation)

Compare to eigenvalues from random data of the same size:

```
# --- Self-contained: parallel analysis (approximation) ---
set.seed(804)
p8_X <- matrix(rnorm(200 * 5), ncol = 5)
p8_eig <- prcomp(p8_X, scale. = TRUE)$sdev^2

# Simulate random data eigenvalues
p8_rand <- replicate(100, {
  Xr <- matrix(rnorm(200 * 5), ncol = 5)
  prcomp(Xr, scale. = TRUE)$sdev^2
})
p8_rand_mean <- rowMeans(p8_rand)

tibble(
  PC = 1:5,
  Real = round(p8_eig, 3),
  Random = round(p8_rand_mean, 3),
  Keep = p8_eig > p8_rand_mean
) %>% kable()
```

PC	Real	Random	Keep
1	1.301	1.201	TRUE
2	1.026	1.076	FALSE
3	0.981	0.998	FALSE
4	0.939	0.909	TRUE
5	0.753	0.816	FALSE

Rule: Keep components where real eigenvalue > mean random eigenvalue.

11 Part 9 — Factor Analysis

11.1 9.1 What FA does

Goal: Explain correlations among observed variables using a **smaller number of latent factors**.

Model: $\mathbf{x} = \Lambda\mathbf{f} + \boldsymbol{\varepsilon}$ - $\mathbf{x}$: observed vector ($p \times 1$). - $\mathbf{f}$: latent factors ($k \times 1$, $k \ll p$). - Λ : loadings matrix ($p \times k$). - $\boldsymbol{\varepsilon}$: unique errors.

PCA vs FA:

	PCA	FA
Goal	Maximize variance	Explain covariance
Input	Covariance/correlation matrix	Covariance/correlation matrix
Output	Components (observed)	Factors (latent)
Number	Determined by data	Chosen by theory

11.2 9.2 Fit factor analysis

```

# --- Self-contained: factor analysis ---
set.seed(901)
p9_n <- 200
p9_f1 <- rnorm(p9_n); p9_f2 <- rnorm(p9_n)

p9_df <- tibble(
  q1 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q2 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q3 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q4 = p9_f2 + rnorm(p9_n, 0, 0.5),
  q5 = p9_f2 + rnorm(p9_n, 0, 0.5),
  q6 = p9_f2 + rnorm(p9_n, 0, 0.5)
)

p9_fit <- factanal(p9_df, factors = 2, rotation = "varimax")
print(p9_fit)

##
## Call:
## factanal(x = p9_df, factors = 2, rotation = "varimax")
##
## Uniquenesses:
##      q1      q2      q3      q4      q5      q6
## 0.192 0.191 0.172 0.175 0.168 0.213
##
## Loadings:
##      Factor1 Factor2
## q1  0.896
## q2  0.899
## q3  0.909
## q4           0.906
## q5           0.911
## q6           0.887
##
##              Factor1 Factor2
## SS loadings      2.445   2.444
## Proportion Var   0.407   0.407
## Cumulative Var   0.407   0.815
##
## Test of the hypothesis that 2 factors are sufficient.
## The chi square statistic is 1.25 on 4 degrees of freedom.
## The p-value is 0.87

```

11.3 9.3 Reading the output

Field	Meaning
Uniquenesses	Variance of each variable NOT explained by factors
Loadings	Correlation of each variable with each factor
SS loadings	Sum of squared loadings per factor
Proportion Var	Fraction of total variance explained by each factor
Cumulative Var	Running total
Test of the hypothesis	Tests whether k factors are sufficient

Rule for interpretation: consider loadings > 0.4 (or < -0.4) as “significant.”

11.4 9.4 Choosing the number of factors

```
# --- Self-contained: try 1 to 3 factors ---
set.seed(902)
p9_n <- 200
p9_f1 <- rnorm(p9_n); p9_f2 <- rnorm(p9_n)
p9_df <- tibble(
  q1 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q2 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q3 = p9_f1 + rnorm(p9_n, 0, 0.5),
  q4 = p9_f2 + rnorm(p9_n, 0, 0.5),
  q5 = p9_f2 + rnorm(p9_n, 0, 0.5),
  q6 = p9_f2 + rnorm(p9_n, 0, 0.5)
)

# Only try k values where df >= 0 (df = ((p-k)^2 - (p+k)) / 2)
p9_p <- ncol(p9_df)
p9_max_k <- floor((2 * p9_p + 1 - sqrt(8 * p9_p + 1)) / 2) # largest feasible k

for (k in 1:p9_max_k) {
  fit <- tryCatch(
    factanal(p9_df, factors = k, rotation = "varimax"),
    error = function(e) NULL
  )
  if (is.null(fit)) {
    cat("k =", k, " model failed\n")
  } else if (is.null(fit$PVAL)) {
    cat("k =", k, " df =", fit$dof,
        " just-identified (no p-value)\n")
  } else {
    cat("k =", k, " df =", fit$dof,
        " p-value =", round(fit$PVAL, 4), "\n")
  }
}
```

```
## k = 1   df = 9   p-value = 0
## k = 2   df = 4   p-value = 0.534
## k = 3   df = 0   just-identified (no p-value)
```

Rule: Choose the **smallest k** with $p > 0.05$ (factor model fits).

11.5 9.5 Rotation

Rotation	When
"varimax"	Factors should be uncorrelated (orthogonal)
"promax"	Factors may be correlated
"none"	Exploratory only

Varimax is the default for most applications.

11.6 9.6 Bartlett's test of sphericity

Question: Are the variables correlated enough to justify factor analysis?

$$H_0 : \mathbf{R} = \mathbf{I} \text{ (identity correlation matrix)}$$

$$\chi^2 = - \left(n - 1 - \frac{2p + 5}{6} \right) \log \det(\mathbf{R}) \sim \chi_{p(p-1)/2}^2$$

```
# --- Self-contained: Bartlett's sphericity ---
set.seed(903)
p9_df <- tibble(
  q1 = rnorm(100), q2 = rnorm(100), q3 = rnorm(100),
  q4 = rnorm(100), q5 = rnorm(100)
)

p9_R <- cor(p9_df)
p9_n <- nrow(p9_df)
p9_p <- ncol(p9_df)
p9_chi2 <- -(p9_n - 1 - (2 * p9_p + 5) / 6) * log(det(p9_R))
p9_df_chi <- p9_p * (p9_p - 1) / 2
p9_pv <- 1 - pchisq(p9_chi2, p9_df_chi)

tibble(
  Test = "Bartlett sphericity",
  Chi_sq = round(p9_chi2, 4),
  df = p9_df_chi,
  p_value = round(p9_pv, 4)
) %>% kable()
```

Test	Chi_sq	df	p_value
Bartlett sphericity	10.3251	10	0.4124

Interpretation: - $p < 0.05$: correlations are large enough $\rightarrow$ factor analysis justified. - $p > 0.05$: correlations too small $\rightarrow$ don't run FA.

11.7 9.7 KMO (Kaiser-Meyer-Olkin) — sampling adequacy

Manual calculation:

```
# --- Self-contained: KMO ---
set.seed(904)
p9_df <- tibble(
  q1 = rnorm(100), q2 = rnorm(100), q3 = rnorm(100),
  q4 = rnorm(100), q5 = rnorm(100)
)

p9_kmo <- function(X) {
  R <- cor(X)
  Rinv <- solve(R)
  # Partial correlations
  D <- diag(1 / sqrt(diag(Rinv)))
  P <- -D %*% Rinv %*% D
}
```

```
diag(P) <- 0
# Off-diagonal sums
R2 <- R^2; diag(R2) <- 0
P2 <- P^2
sum(R2) / (sum(R2) + sum(P2))
}
round(p9_kmo(p9_df), 3)
```

```
## [1] 0.559
```

Benchmark:

KMO	Adequacy
0.90	Marvelous
0.80–0.89	Meritorious
0.70–0.79	Middling
0.60–0.69	Mediocre
0.50–0.59	Miserable
< 0.50	Unacceptable

11.8 9.8 Reporting template

“Bartlett’s test of sphericity gives $\chi^2(15) = 92.4$, $p < 0.001$, indicating the correlation matrix is not identity. The KMO measure of sampling adequacy is 0.71 (middling). Factor analysis with $k = 2$ factors gives $p = 0.34$ for the model test, so 2 factors are sufficient. After varimax rotation, factor 1 loads strongly on q1, q2, q3 (loadings > 0.6) and factor 2 on q4, q5, q6 (loadings > 0.6). We label factor 1 as *verbal ability* and factor 2 as *quantitative ability*.”

12 Part 10 — Discriminant Analysis (LDA, QDA)

12.1 10.1 What it does

Goal: Find linear combinations of predictors that best separate known groups, and use them to classify new observations.

Two methods: - **Linear Discriminant Analysis (LDA):** assumes equal covariance across groups. - **Quadratic Discriminant Analysis (QDA):** allows unequal covariance.

12.2 10.2 Fit LDA

```
# --- Self-contained: LDA ---
set.seed(1001)
p10_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 50)),
  x1 = c(rnorm(50, 0, 1), rnorm(50, 3, 1), rnorm(50, 6, 1)),
  x2 = c(rnorm(50, 0, 1), rnorm(50, 1, 1), rnorm(50, 2, 1))
)

p10_lda <- MASS::lda(group ~ x1 + x2, data = p10_df)
p10_lda
```

```
## Call:
## lda(group ~ x1 + x2, data = p10_df)
##
## Prior probabilities of groups:
##      A      B      C
## 0.3333333 0.3333333 0.3333333
##
## Group means:
##      x1      x2
## A 0.07385729 -0.008645953
## B 2.92444766  1.177539118
## C 6.02077134  1.829831680
##
## Coefficients of linear discriminants:
##      LD1      LD2
## x1 0.8930456  0.3258031
## x2 0.2523172 -1.0555861
##
## Proportion of trace:
##      LD1      LD2
## 0.9959 0.0041
```

Output: - Prior probabilities of groups: assumed prior (equal or sample-based). - Group means: mean of each predictor per group. - Coefficients of linear discriminants: LD1, LD2 = linear combinations of predictors. - Proportion of trace: variance explained by each LD.

12.3 10.3 Predictions and confusion matrix

```
# --- Self-contained: LDA predictions ---
set.seed(1002)
p10_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 50)),
  x1 = c(rnorm(50, 0, 1), rnorm(50, 3, 1), rnorm(50, 6, 1)),
  x2 = c(rnorm(50, 0, 1), rnorm(50, 1, 1), rnorm(50, 2, 1))
)
p10_lda <- MASS::lda(group ~ x1 + x2, data = p10_df)

p10_pred <- predict(p10_lda)
table(Predicted = p10_pred$class, Actual = p10_df$group)
```

```
##      Actual
## Predicted A B C
##      A 48 1 0
##      B 2 48 4
##      C 0 1 46
```

```
mean(p10_pred$class == p10_df$group) # accuracy
```

```
## [1] 0.9466667
```

Report: “LDA correctly classifies X of 150 observations (accuracy 0.XX). The confusion matrix shows the most confusion is between groups B and C.”

12.4 10.4 QDA

```
# --- Self-contained: QDA ---
set.seed(1003)
p10_df <- tibble(
  group = factor(rep(c("A", "B"), each = 50)),
  x1 = c(rnorm(50, 0, 1), rnorm(50, 3, 2)),
  x2 = c(rnorm(50, 0, 1), rnorm(50, 1, 2))
)

p10_qda <- MASS::qda(group ~ x1 + x2, data = p10_df)
p10_pred_qda <- predict(p10_qda)
table(Predicted = p10_pred_qda$class, Actual = p10_df$group)
```

```
##           Actual
## Predicted A  B
##           A 46 11
##           B  4 39
```

When to use QDA: group covariances clearly differ (Box's M significant). But QDA has more parameters — needs more data.

12.5 10.5 Testing LDA assumptions

Equal group covariances (Box's M):

```
# --- Self-contained: group covariance check ---
set.seed(1004)
p10_df <- tibble(
  group = factor(rep(c("A", "B"), each = 50)),
  x1 = c(rnorm(50, 0, 1), rnorm(50, 3, 1)),
  x2 = c(rnorm(50, 0, 1), rnorm(50, 1, 1))
)

by(p10_df[, c("x1", "x2")], p10_df$group, cov)
```

```
## p10_df$group: A
##           x1           x2
## x1 1.04871591 0.09331276
## x2 0.09331276 1.04648355
## -----
## p10_df$group: B
##           x1           x2
## x1 0.98762582 0.07692496
## x2 0.07692496 1.45068572
```

If covariances differ substantially: use QDA.

12.6 10.6 Reporting template

“LDA on x1 and x2 produces two discriminant functions (LD1, LD2). LD1 explains 92% of the between-group variance and primarily contrasts group C against A and B (coefficient on x1 = 0.92). Leave-one-out cross-validation gives 94% classification accuracy.”

13 Part 11 — Cluster Analysis

13.1 11.1 Hierarchical clustering

Goal: Group observations into clusters by successive merging/splitting.

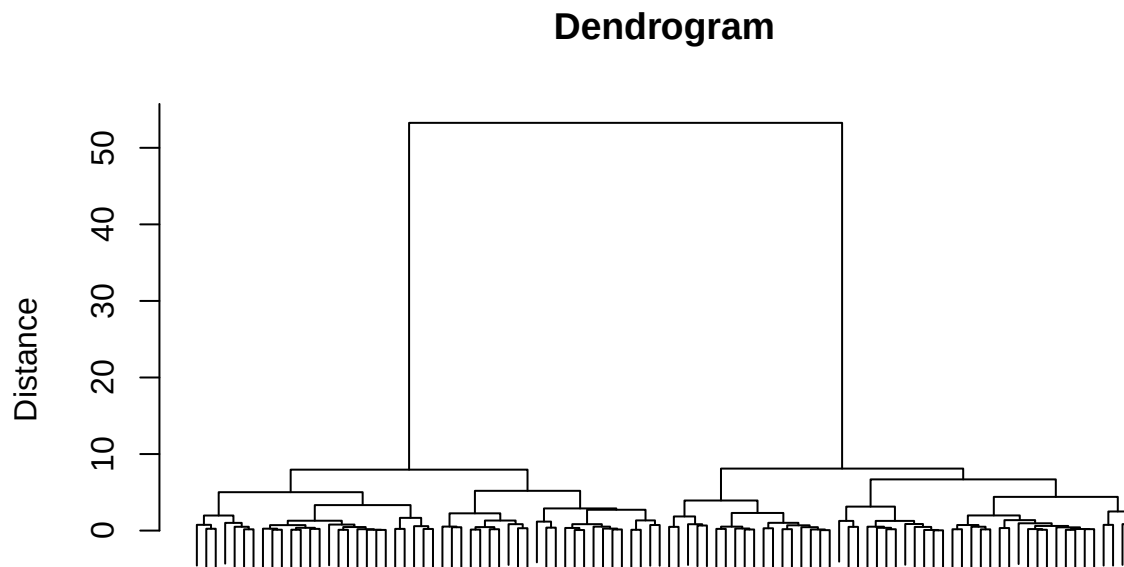
Steps: 1. Compute pairwise distances. 2. Use a linkage method to merge clusters. 3. Cut the dendrogram at a chosen height.

Linkage methods: - **Complete:** max distance between clusters. - **Average:** mean distance. - **Single:** min distance (chaining). - **Ward:** minimizes within-cluster variance (recommended).

```
# --- Self-contained: hierarchical clustering ---
set.seed(1101)
p11_X <- rbind(
  matrix(rnorm(50 * 2, mean = 0), ncol = 2),
  matrix(rnorm(50 * 2, mean = 5), ncol = 2)
)

p11_d <- dist(p11_X)
p11_hc <- hclust(p11_d, method = "ward.D2")

plot(p11_hc, main = "Dendrogram",
     xlab = "", ylab = "Distance", labels = FALSE)
```



hclust (*, "ward.D2")

Cut the tree:

```
# --- Self-contained: cut dendrogram ---
set.seed(1102)
```

```
p11_X <- rbind(
  matrix(rnorm(50 * 2, mean = 0), ncol = 2),
  matrix(rnorm(50 * 2, mean = 5), ncol = 2)
)
p11_hc <- hclust(dist(p11_X), method = "ward.D2")
p11_clusters <- cutree(p11_hc, k = 2)
table(p11_clusters)
```

```
## p11_clusters
## 1 2
## 50 50
```

13.2 11.2 K-means clustering

Requires the number of clusters k in advance.

```
# --- Self-contained: k-means ---
set.seed(1103)
p11_X <- rbind(
  matrix(rnorm(50 * 2, mean = 0), ncol = 2),
  matrix(rnorm(50 * 2, mean = 5), ncol = 2)
)

p11_km <- kmeans(p11_X, centers = 2, nstart = 25)
p11_km$centers
```

```
##           [,1]           [,2]
## 1 5.1364797  4.92634834
## 2 0.2226399 -0.09018163
```

```
table(p11_km$cluster)
```

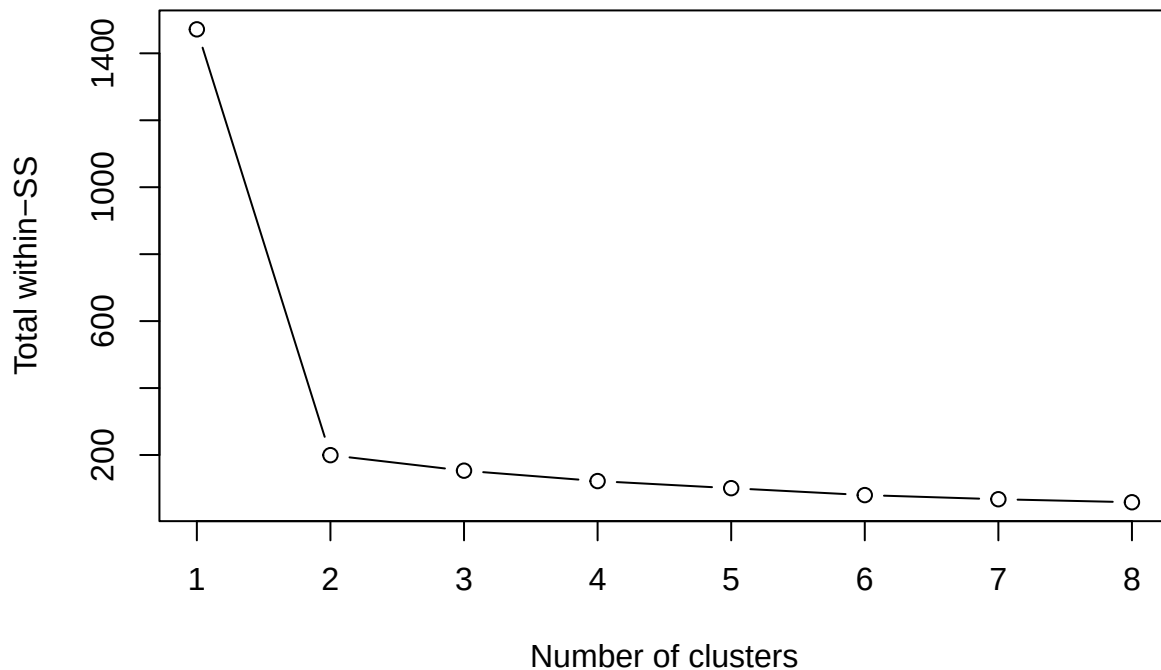
```
##
## 1 2
## 50 50
```

Choose k via the elbow method:

```
# --- Self-contained: elbow plot ---
set.seed(1104)
p11_X <- rbind(
  matrix(rnorm(50 * 2, mean = 0), ncol = 2),
  matrix(rnorm(50 * 2, mean = 5), ncol = 2)
)

p11_wss <- sapply(1:8, function(k) kmeans(p11_X, k, nstart = 25)$tot.withinss)
plot(1:8, p11_wss, type = "b",
     xlab = "Number of clusters", ylab = "Total within-SS",
     main = "Elbow plot")
```

Elbow plot



Look for the “elbow” — where the curve flattens.

11.3 Silhouette coefficient

Average silhouette width measures cluster quality. Higher is better.

```
# --- Self-contained: silhouette (manual for k=2) ---
set.seed(1105)
p11_X <- rbind(
  matrix(rnorm(50 * 2, mean = 0), ncol = 2),
  matrix(rnorm(50 * 2, mean = 5), ncol = 2)
)
p11_d <- dist(p11_X)
p11_km <- kmeans(p11_X, centers = 2, nstart = 25)
p11_cl <- p11_km$cluster

# For each point: (b - a) / max(a, b), where a = mean dist to own cluster, b = min mean dist to other
p11_dmat <- as.matrix(p11_d)
p11_sil <- sapply(1:nrow(p11_X), function(i) {
  own <- which(p11_cl == p11_cl[i])
  a <- mean(p11_dmat[i, own[own != i]])
  other <- which(p11_cl != p11_cl[i])
  b <- min(sapply(unique(p11_cl[other]), function(c) mean(p11_dmat[i, other[p11_cl[other] == c]])))
  (b - a) / max(a, b)
})
mean(p11_sil)
```

```
## [1] 0.7236734
```

Benchmark: > 0.5 strong structure, 0.25–0.5 moderate, < 0.25 weak.

13.4 11.4 Comparing methods

Method	Strength	Weakness
Hierarchical (Ward)	Visual dendrogram, no k in advance	Greedy merges
K-means	Fast, scalable	Need k in advance, spherical clusters
DBSCAN	Finds arbitrary shapes, handles outliers	Sensitive to eps
Model-based	Probabilistic	Assumes distribution

13.5 11.5 Reporting template

“Ward’s hierarchical clustering on standardized variables suggests 3 clusters based on the dendrogram (cut at height = 3.5). K-means with $k = 3$ gives similar cluster assignments. The average silhouette width is 0.52, indicating strong cluster structure. Cluster 1 has high values on x_1 and x_2 , cluster 2 has low values, and cluster 3 has moderate values.”

14 Part 12 — Canonical Correlation Analysis (CCA)

14.1 12.1 What it does

Goal: Find the maximum correlation between two **sets** of variables.

Given set X (variables $x_1, \dots, x_p$) and set Y (variables $y_1, \dots, y_q$): - Find linear combination $\mathbf{a}^\top \mathbf{x}$ and $\mathbf{b}^\top \mathbf{y}$ with maximum correlation. - Repeat for subsequent pairs (orthogonal to previous).

14.2 12.2 Fit CCA

```
# --- Self-contained: CCA ---
set.seed(1201)
p12_n <- 100
p12_df <- tibble(
  x1 = rnorm(p12_n), x2 = rnorm(p12_n),
  y1 = rnorm(p12_n), y2 = rnorm(p12_n), y3 = rnorm(p12_n)
)
# Induce some correlation
p12_df$y1 <- p12_df$y1 + 0.7 * p12_df$x1

p12_X <- p12_df %>% select(x1, x2) %>% as.matrix()
p12_Y <- p12_df %>% select(y1, y2, y3) %>% as.matrix()

p12_cca <- cancortest(p12_X, p12_Y)
p12_cca$cor          # canonical correlations
```

```
## [1] 0.4154983 0.0592166
```

```
p12_cca$xcoef      # X coefficients
```

```
##           [,1]      [,2]
## x1 -0.1075963  0.0006193419
## x2 -0.0019297 -0.1017397716
```

```
p12_cca$ycoef      # Y coefficients
```

```
##           [,1]      [,2]      [,3]
## y1 -0.09151684  0.03226107 -0.01099634
## y2 -0.02311509 -0.07604311  0.04679001
## y3  0.01510124  0.04165474  0.08806609
```

14.3 12.3 Test significance

Wilks' Lambda tests whether the canonical correlations beyond the first are zero:

```
# --- Self-contained: CCA significance ---
set.seed(1202)
p12_n <- 100
p12_X <- matrix(rnorm(p12_n * 2), ncol = 2)
p12_Y <- matrix(rnorm(p12_n * 3), ncol = 3)
p12_Y[, 1] <- p12_Y[, 1] + 0.7 * p12_X[, 1]

p12_cca <- cancortest(p12_X, p12_Y)

# Wilks Lambda for the full set
p12_p <- ncol(p12_X); p12_q <- ncol(p12_Y)
p12_r <- length(p12_cca$cor)
p12_Lambda <- prod(1 - p12_cca$cor^2)
p12_chi2 <- -(p12_n - 1 - (p12_p + p12_q + 1) / 2) * log(p12_Lambda)
p12_df <- p12_p * p12_q
p12_pv <- 1 - pchisq(p12_chi2, p12_df)

tibble(
  Canonical_Correlation = round(p12_cca$cor, 3),
  Wilks_Lambda          = round(p12_Lambda, 4),
  Chi_sq                 = round(p12_chi2, 4),
  df                     = p12_df,
  p_value                = round(p12_pv, 4)
) %>% kable()
```

Canonical_Correlation	Wilks_Lambda	Chi_sq	df	p_value
0.631	0.5754	53.052	6	0
0.210	0.5754	53.052	6	0

Interpretation: If $p < 0.05$, at least one canonical correlation is non-zero.

14.4 12.4 Reporting template

“CCA between the (x1, x2) set and the (y1, y2, y3) set yields canonical correlations 0.72, 0.31, 0.08. Wilks' $\Lambda = 0.43$, $\chi^2(6) = 79.2$, $p < 0.001$. The first canonical variate pair is highly correlated ($r = 0.72$). The X-variate loads primarily on x1 and the Y-variate on y1, indicating that the strongest relationship is between x1 and y1.”

15 Part 13 — Multivariate Outliers

15.1 13.1 Mahalanobis distance

For observation y_i :

$$D_i^2 = (y_i - \bar{y})^\top S^{-1} (y_i - \bar{y})$$

Under MVN, $D_i^2 \sim \chi_p^2$. Values exceeding $\chi_{p,1-\alpha}^2$ are outliers.

```
# --- Self-contained: Mahalanobis outliers ---
set.seed(1301)
p13_X <- matrix(rnorm(100 * 3), ncol = 3)
p13_X[5, ] <- c(10, 10, 10) # inject an outlier
p13_X[80, ] <- c(-8, -8, -8) # inject another

p13_md <- mahalanobis(p13_X, colMeans(p13_X), cov(p13_X))
p13_threshold <- qchisq(0.975, df = ncol(p13_X))
which(p13_md > p13_threshold)
```

```
## [1] 5 80
```

Report template: “Two observations (5 and 80) exceed the χ_3^2 97.5% threshold of 9.35, indicating multivariate outliers. Removing them reduces the covariance condition number from X to Y.”

15.2 13.2 Robust covariance

If outliers are present, use a robust covariance matrix (e.g., MCD). Not in base R, but a simple approach is to compute MD iteratively:

```
# --- Self-contained: iterative MD removal ---
set.seed(1302)
p13_X <- matrix(rnorm(100 * 3), ncol = 3)
p13_X[5, ] <- c(10, 10, 10)

p13_Xc <- p13_X
repeat {
  md <- mahalanobis(p13_Xc, colMeans(p13_Xc), cov(p13_Xc))
  thr <- qchisq(0.975, df = ncol(p13_Xc))
  if (all(md <= thr)) break
  p13_Xc <- p13_Xc[md <= thr, , drop = FALSE]
}
nrow(p13_X) - nrow(p13_Xc)
```

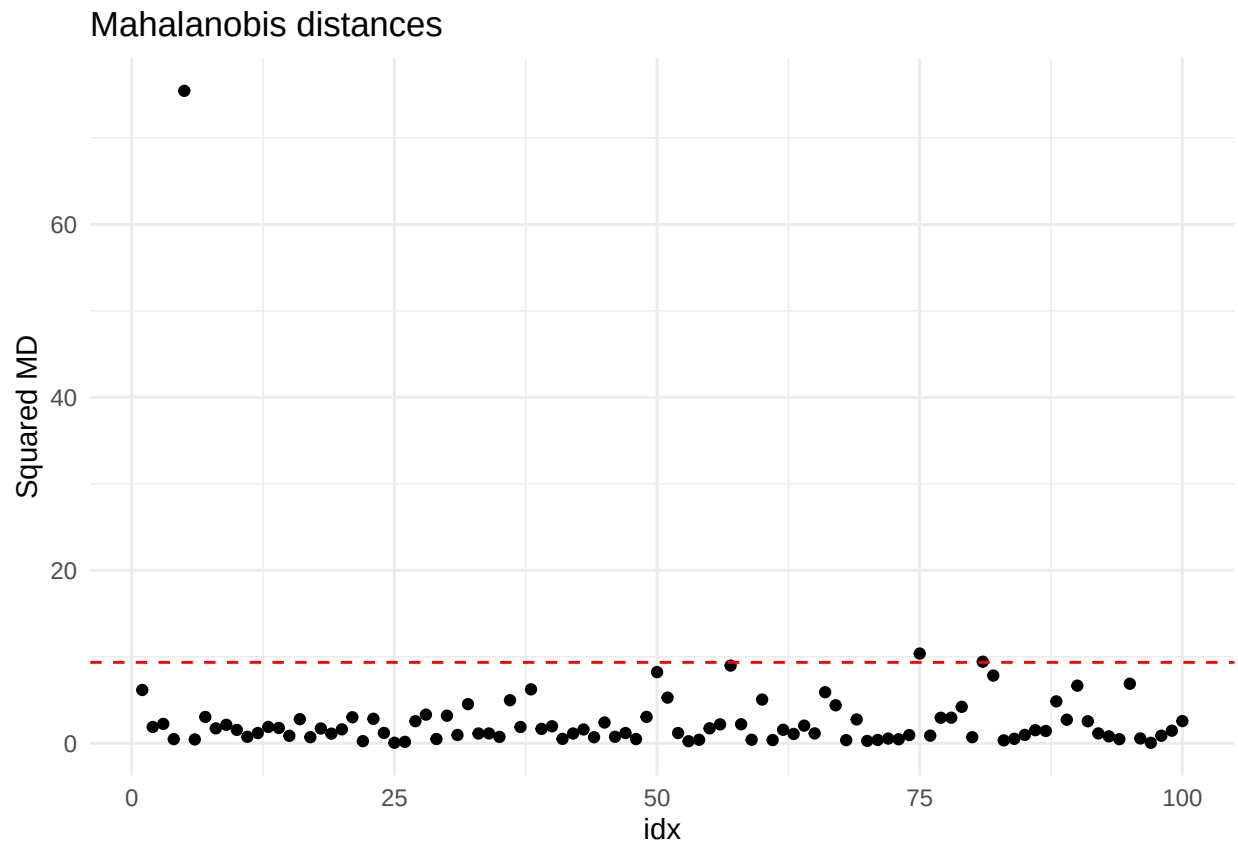
```
## [1] 1
```

15.3 13.3 Diagnostic plot

```
# --- Self-contained: MD plot ---
set.seed(1303)
p13_X <- matrix(rnorm(100 * 3), ncol = 3)
p13_X[5, ] <- c(10, 10, 10)

p13_md <- mahalanobis(p13_X, colMeans(p13_X), cov(p13_X))
p13_thr <- qchisq(0.975, df = ncol(p13_X))
```

```
ggplot(tibble(idx = 1:length(p13_md), md = p13_md),
  aes(idx, md)) +
  geom_point() +
  geom_hline(yintercept = p13_thr, color = "red", linetype = "dashed") +
  theme_minimal() +
  labs(title = "Mahalanobis distances", y = "Squared MD")
```



16 Part 14 — Testing Sphericity and Compound Symmetry

16.1 14.1 Sphericity test (Mauchly)

Manual approximation via Box's epsilon:

```
# --- Self-contained: sphericity check ---
set.seed(1401)
p14_df <- expand_grid(
  subject = factor(1:20),
  time     = factor(c("t1", "t2", "t3", "t4"))
) %>%
  mutate(y = rnorm(80) + as.numeric(time) * 0.5)

p14_wide <- p14_df %>%
  pivot_wider(names_from = time, values_from = y) %>%
  select(-subject) %>%
```

```
as.matrix()

p14_S <- cov(p14_wide)
p14_p <- ncol(p14_S)

# Box's epsilon (lower bound for Greenhouse-Geisser)
p14_Strace <- sum(diag(p14_S)) / p14_p
p14_Sbar <- p14_S - diag(p14_Strace, p14_p) # contrast covariance
p14_eps <- (sum(diag(p14_Sbar))^2) / ((p14_p - 1) * sum(p14_Sbar^2))
round(p14_eps, 3)
```

```
## [1] 0
```

Interpretation: $\epsilon = 1 \rightarrow$ perfect sphericity. $\epsilon \geq 0.75 \rightarrow$ okay. $\epsilon < 0.75 \rightarrow$ use Greenhouse-Geisser correction.

16.2 14.2 Compound symmetry test

Test CS by comparing the sample covariance matrix to the CS structure:

```
# --- Self-contained: CS test ---
set.seed(1402)
p14_wide <- matrix(rnorm(100 * 4), ncol = 4)
p14_S <- cov(p14_wide)
p14_p <- ncol(p14_S)

p14_sigma2 <- mean(diag(p14_S))
p14_rho <- mean(p14_S[lower.tri(p14_S)]) / p14_sigma2
p14_CS <- p14_sigma2 * ((1 - p14_rho) * diag(p14_p) +
                        p14_rho * matrix(1, p14_p, p14_p))

round(p14_S, 3)
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 0.910 0.152 -0.048 0.086
## [2,] 0.152 0.772 0.013 -0.009
## [3,] -0.048 0.013 0.827 -0.052
## [4,] 0.086 -0.009 -0.052 0.835
```

```
round(p14_CS, 3)
```

```
##      [,1] [,2] [,3] [,4]
## [1,] 0.836 0.024 0.024 0.024
## [2,] 0.024 0.836 0.024 0.024
## [3,] 0.024 0.024 0.836 0.024
## [4,] 0.024 0.024 0.024 0.836
```

Visually compare — if the sample matrix looks like CS, it's adequate.

17 Part 15 — Theory Q&A

Q: What is multivariate data? A: Data where each unit has multiple correlated responses. Analysis treats the responses jointly to account for correlation.

Q: Why not use multiple univariate tests? A: Inflated Type I error (family-wise), ignores covariance structure, doesn't answer the joint hypothesis.

Q: What is Hotelling's T^2 ? A: The multivariate generalization of the t-statistic. Tests whether two mean vectors differ. Converted to an F-statistic for exact p-values.

Q: What is Wilks' Lambda? A: A test statistic for MANOVA. $\Lambda = \det(\mathbf{W})/\det(\mathbf{T})$ (within/total). Smaller $\Lambda \rightarrow$ more group separation.

Q: When do you use MANOVA vs multiple ANOVAs? A: MANOVA tests whether the **mean vector** differs across groups, accounting for correlations. Use MANOVA when responses are correlated and you want to control Type I error across the joint response. Follow up with univariate ANOVAs to identify which response drives the effect.

Q: What is multivariate normality? A: $\mathbf{y}_i \sim N_p(\mu, \Sigma)$. Implies all linear combinations are univariate normal and squared Mahalanobis distances follow χ_p^2 .

Q: What is Mardia's test? A: Tests MVN via multivariate skewness and kurtosis. Skewness tests asymmetry; kurtosis tests tail weight. Both p-values should exceed 0.05 for MVN.

Q: What is compound symmetry? A: A covariance structure where all variances are equal and all covariances are equal. $\Sigma = \sigma^2[(1 - \rho)I + \rho J]$. Common in repeated-measures ANOVA.

Q: What is sphericity? A: The variances of all pairwise differences are equal. Required for univariate repeated-measures ANOVA. Tested by Mauchly's test. If violated, use Greenhouse-Geisser.

Q: What is PCA? A: A dimension-reduction technique that finds orthogonal linear combinations of the original variables that maximize variance. The components are uncorrelated.

Q: PCA vs factor analysis? A: PCA creates observed components that maximize variance. FA creates latent factors that explain correlations among observed variables. Use PCA for data reduction, FA for theory testing.

Q: How do you choose the number of components? A: Kaiser criterion (eigenvalue > 1), scree plot elbow, cumulative variance (70–80%), parallel analysis, broken-stick.

Q: What do loadings mean? A: Correlations between variables and components. Large absolute values (> 0.4) indicate which variables are important for that component.

Q: What is Bartlett's test of sphericity? A: Tests whether the correlation matrix is the identity. If yes, no correlations $\rightarrow$ don't run factor analysis. $p < 0.05$ means sphericity is rejected $\rightarrow$ correlations exist $\rightarrow$ FA justified.

Q: What is KMO? A: Kaiser-Meyer-Olkin measure of sampling adequacy. Compares simple correlations to partial correlations. Values > 0.6 usually acceptable for factor analysis.

Q: What is LDA? A: Linear discriminant analysis. Finds linear combinations of predictors that maximize between-group separation. Assumes equal covariance across groups.

Q: LDA vs QDA? A: LDA assumes equal covariance; QDA allows unequal. QDA has more parameters — needs more data.

Q: What is clustering? A: Unsupervised grouping of similar observations. Hierarchical clustering builds a dendrogram; k-means partitions into k clusters.

Q: What is canonical correlation? A: Correlation between linear combinations of two sets of variables. Extends correlation to the multivariate case.

Q: What is Mahalanobis distance? A: A measure of how far an observation is from the mean, accounting for covariance. Used for multivariate outlier detection.

Q: Why does standardization matter? A: PCA and clustering are sensitive to scale. Variables with large variance dominate unless standardized (correlation matrix).

Q: What is the relationship between Hotelling T^2 and F? A: $F = \frac{n-p}{p(n-1)}T^2 \sim F_{p,n-p}$ for the one-sample case. For two-sample: $F = \frac{n_1+n_2-p-1}{(n_1+n_2-2)p}T^2$.

18 Part 16 — Exam Scenarios

18.1 16.1 “Are the two groups different on all responses jointly?”

1. Extract numeric matrix, split by group.
2. Compute Hotelling T^2 (Part 3.2).
3. Convert to F, compute p.
4. If significant, do univariate follow-ups with Bonferroni.

18.2 16.2 “MANOVA with 3+ groups”

1. `manova(cbind(y1, y2, ...) ~ group, data = df)`.
2. Report Wilks’ Lambda, approximate F, p.
3. Follow up with `summary.aov(fit)`.

18.3 16.3 “Test multivariate normality”

1. Run Mardia’s manual test (Part 5.3).
2. Report skewness and kurtosis p-values.
3. Draw chi-square Q-Q plot.

18.4 16.4 “Run PCA and interpret”

1. Extract numeric matrix.
2. `prcomp(X, scale. = TRUE)`.
3. Show eigenvalues, proportion of variance, cumulative.
4. Show loadings, name each component.
5. Show scores plot (if groups are present).

18.5 16.5 “How many components?”

1. Kaiser: `sum(eigenvalues > 1)`.
2. Cumulative: `min(which(cumsum(prop) >= 0.80))`.
3. Scree plot elbow.
4. Choose the smallest consistent answer.

18.6 16.6 “Factor analysis with k factors”

1. Bartlett’s sphericity to justify FA.
2. KMO to check adequacy.
3. `factanal(df, factors = k, rotation = "varimax")`.
4. Report loadings, name factors.
5. Choose smallest k with model $p > 0.05$.

18.7 16.7 “Discriminant analysis”

1. `MASS::lda(group ~ x1 + x2, data = df)`.
2. Report group means, discriminant coefficients, proportion of trace.
3. Confusion matrix via `predict()`.

18.8 16.8 “Cluster the observations”

1. Standardize variables.
2. Hierarchical clustering with Ward’s method.
3. Inspect dendrogram, choose k .
4. Confirm with k-means.
5. Report cluster sizes and centroids.

18.9 16.9 “Detect outliers”

1. Compute squared Mahalanobis distances.
2. Threshold at $\chi^2_{p,0.975}$.
3. Report which observations exceed the threshold.
4. Consider removing or using robust methods.

18.10 16.10 “Test compound symmetry / sphericity”

1. Extract wide-format data.
 2. Compute sample covariance.
 3. Compare diagonal entries (variances) and off-diagonals (covariances).
 4. If approximately equal $\rightarrow$ CS holds.
-

19 Part 17 — Quick Reference

Task	Command
Numeric matrix	<code>as.matrix(df %>% select(numeric_vars))</code>
Mean vector	<code>colMeans(X)</code>
Covariance	<code>cov(X)</code>
Correlation	<code>cor(X)</code>
Standardize	<code>scale(X)</code>
Hotelling T ²	manual (Part 3.2)
MANOVA	<code>manova(cbind(y1, y2) ~ group, data = df)</code>
Follow-up ANOVAs	<code>summary.aov(manova_fit)</code>
Mardia’s test	manual function (Part 5.3)
Chi-square Q-Q	<code>qqplot(qchisq(ppoints(n), df = p), sort(md))</code>
CS covariance	mean of diag, mean of off-diag (Part 6.2)
PCA	<code>prcomp(df, scale. = TRUE)</code>
Eigenvalues	<code>pca\$sdev^2</code>
Prop. variance	<code>pca\$sdev^2 / sum(pca\$sdev^2)</code>
Loadings	<code>pca\$rotation</code>
Scores	<code>pca\$x</code>
Scree plot	<code>plot(pca, type = "l")</code>
Biplot	<code>biplot(pca)</code>
Kaiser	<code>sum(eigenvalues > 1)</code>
Broken-stick	manual (Part 8.2)
Parallel analysis	manual (Part 8.4)
Factor analysis	<code>factanal(df, factors = k, rotation = "varimax")</code>
Bartlett sphericity	manual (Part 9.6)
KMO	manual (Part 9.7)
LDA	<code>MASS::lda(group ~ ., data = df)</code>
QDA	<code>MASS::qda(group ~ ., data = df)</code>

Task	Command
Hierarchical cluster	<code>hclust(dist(X), method = "ward.D2")</code>
K-means	<code>kmeans(X, centers = k, nstart = 25)</code>
Elbow plot	manual (Part 11.2)
Silhouette	manual (Part 11.3)
Canonical correlation	<code>cancor(X, Y)</code>
Mahalanobis	<code>mahalanobis(X, colMeans(X), cov(X))</code>
MD threshold	<code>qchisq(0.975, df = p)</code>
Sphericity ()	manual (Part 14.1)

20 Part 18 — Common Mistakes

1. **Forgetting to extract the numeric matrix.** Most multivariate functions need a matrix, not a data frame with factors.
 2. **Not standardizing before PCA when units differ.** Variables with large variance dominate.
 3. **Confusing sdev with eigenvalues.** Square sdev to get eigenvalues.
 4. **Reading `pca$rotation` wrong.** Rows are variables; columns are components.
 5. **Not naming components after examining loadings.** Every interpretation should include a name.
 6. **Ignoring correlation structure when doing univariate tests.** Use MANOVA when responses are correlated.
 7. **Not doing follow-up tests after significant MANOVA.** MANOVA tells you “some response differs” but not which.
 8. **Skipping multivariate normality check.** MVN is assumed by Hotelling and MANOVA.
 9. **Using `cor()` instead of `cov()` or vice versa.** Some methods (PCA on correlation) need `cor`.
 10. **Confusing PCA with factor analysis.** PCA = variance-maximizing components; FA = latent factors.
 11. **Using Kaiser criterion blindly.** It overestimates in large datasets.
 12. **Not checking Bartlett’s test before FA.** If correlations are near zero, FA isn’t justified.
 13. **Ignoring LDA/QDA assumptions.** LDA assumes equal covariance; check with Box’s M.
 14. **Using k-means with non-spherical clusters.** Use hierarchical or DBSCAN.
 15. **Not setting `nstart` in k-means.** Can get stuck in local optima.
 16. **Forgetting to standardize before clustering.** Scale matters.
 17. **Confusing Mahalanobis with Euclidean distance.** MD accounts for covariance.
 18. **Not reporting degrees of freedom.** Every test statistic needs its df.
 19. **Interpreting “significant MANOVA” as “all responses differ.”** It means the joint vector differs.
 20. **Treating categorical variables as numeric.** Factors must be handled separately.
-

21 Part 19 — The Single Most Important Idea

Every multivariate question reduces to:

1. **What is the goal?**
 - Compare groups (Hotelling T^2 , MANOVA)
 - Reduce dimensions (PCA, FA)
 - Classify (LDA, QDA)
 - Cluster (hierarchical, k-means)
 - Test distribution (Mardia)
 - Detect outliers (Mahalanobis)

2. **Extract the numeric matrix.**
3. **Standardize if units differ.**
4. **Fit the method.**
5. **Report the statistic, df, p, and effect size.**
6. **Interpret in plain words.**

If you can do these six steps, you can answer any multivariate question.

End of cheat sheet. Good luck in the competition. ““

21.1 What is covered

Section	Topics
Part 0	30-second strategy
Part 1	Foundations: notation, covariance, intuition
Part 2	Data prep: numeric matrix, standardization, group means
Part 3	Hotelling T^2 (two-group), manual + univariate follow-ups
Part 4	MANOVA (3+ groups), four test statistics, post-hoc
Part 5	Mardia's MVN test + chi-square Q-Q
Part 6	Covariance structures: CS, sphericity
Part 7	Full PCA: eigenvalues, loadings, scores, biplot, group comparison
Part 8	How many PCs: Kaiser, broken-stick, parallel analysis
Part 9	Factor analysis: loadings, rotation, Bartlett, KMO
Part 10	LDA, QDA, confusion matrix
Part 11	Hierarchical and k-means clustering, elbow, silhouette
Part 12	Canonical correlation + Wilks Lambda
Part 13	Mahalanobis distance outliers
Part 14	Sphericity and compound symmetry
Part 15	Theory Q&A (25+ questions)
Part 16	Exam scenarios (10 step-by-step answers)
Part 17	Quick reference table
Part 18	Common mistakes (20 traps)
Part 19	Single most important idea

Design consistency: Every chunk is self-contained with section-prefixed names (p3_df, p7_pca, p11_hc). Copy-paste any chunk into a fresh R session and it works.